

REACT-TRACE: A Semantics for Understanding React Hooks

An Operational Semantics and a Visualizer for Clarifying React Hooks

JAY LEE, JOONGWON AHN, and KWANGKEUN YI, Seoul National University, Korea

React has become the most widely used web front-end framework, enabling the creation of user interfaces in a declarative and compositional manner. Hooks are a set of APIs that manage side effects in function components in React. However, their semantics are often seen as opaque to developers, leading to UI bugs. We introduce REACT-TRACE, a formalization of the semantics of the essence of React Hooks, providing a semantics that clarifies their behavior. We demonstrate that our model captures the behavior of React, by theoretically showing that it embodies essential properties of Hooks and empirically comparing our REACT-TRACE-definitional interpreter against a test suite. Furthermore, we showcase a practical visualization tool based on the formalization to demonstrate how developers can better understand the semantics of Hooks.

CCS Concepts: • **Theory of computation** → **Operational semantics**; • **Software and its engineering** → **Functional languages**; *Interpreters*; **Graphical user interface languages**; **Formal language definitions**; • **Human-centered computing** → *Visualization systems and tools*.

Additional Key Words and Phrases: React, Hooks, Render semantics, Functional reactive programming

ACM Reference Format:

Jay Lee, Joongwon Ahn, and Kwangkeun Yi. 2025. REACT-TRACE: A Semantics for Understanding React Hooks: An Operational Semantics and a Visualizer for Clarifying React Hooks. *Proc. ACM Program. Lang.* 9, OOPSLA2, Article 289 (October 2025), 42 pages. <https://doi.org/10.1145/3763067>

1 Introduction

1.1 Overview of React

React (react.dev) is the most widely used framework for developing web-based graphical user interfaces (GUI) today [[Stack Exchange, Inc. 2024](https://stackexchange.com)]. React developers structure the UI as a tree of *components*, where each component serves as a *functional* specification that declaratively defines a portion of the UI and its behavior. To declare user interactions in a component, developers use *Hooks*, a collection of library functions that “hooks into” the runtime to manage state and handle side effects within components.

A React component is a function that serves as a declarative specification of a UI, which is read by the React runtime to render the interface. The process of “reading” this specification is nothing special—the runtime directly invokes the component, which is a plain function. When these components are called with appropriate arguments, they return a data structure representing how the component should appear on the screen. Ideally, when a component is called with the same arguments, it should render the same interface [[Staff 2016](https://staff.washington.edu)].

This functional approach differs significantly from traditional object-oriented UI frameworks. In object-oriented frameworks like Flutter (flutter.dev), Qt (qt.io), UIKit (developer.apple.com), or even legacy versions of React [[Madsen et al. 2020](https://madsen.io); [Meta Platforms, Inc. 2025a](https://meta.com)], components are modeled with classes, where each view directly corresponds to an instance of a class. Developers define

Authors’ Contact Information: Jay Lee, jhlee@ropas.snu.ac.kr; Joongwon Ahn, jwahn@ropas.snu.ac.kr; Kwangkeun Yi, kwang@ropas.snu.ac.kr, Department of Computer Science and Engineering, Seoul National University, Seoul, Korea.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2025 Copyright held by the owner/author(s).

ACM 2475-1421/2025/10-ART289

<https://doi.org/10.1145/3763067>

different methods to be called at different lifecycle phases—such as initialization, state update, etc.—creating a clear separation.

React, however, introduces a different model: a single function component plays multiple roles—the same component is called for both creating a new view and updating an existing view. To enable this, React switches between different implementations of the same Hook at runtime depending on the rendering phase.

```
1 function Counter({ x }) {
2   const [s, setS] = useState(() => x);
3   const h = () => setS((s) => s+1);
4   return <button onClick={h}> {s}
5     </button>; }
```

The Counter component on the left demonstrates the **useState** Hook for managing state. This simple component increments its state *s* each time the button is clicked.

When first rendered, **useState** returns an initial value from the initializer function $() \Rightarrow x$. Here, *s* captures the current state, and *setS* accepts an updater to set a new state. In subsequent renders, **useState** automatically “remembers” the previous state and returns it instead. **button** is correctly rendered with new *s*, by efficiently diffing the returned **button**—this is called *reconciliation* [Meta Platforms, Inc. 2025e]. This example illustrates how even a simple component like Counter has different semantics across lifecycle phases.

Another peculiarity is that developers *cannot* use JavaScript (JS) variables and direct assignments to manage a component’s state. In the above example, assigning a new value to *s* does not update the state of Counter—*setS* must be used to update the state.

Moreover, developers must adhere to specific rules imposed by React, collectively known as the *Rules of React* [Meta Platforms, Inc. 2025h]. These rules represent invariants that components must maintain for React to properly manage the UI. The Cond component on the right violates one of these rules on line 4 by calling **useState** conditionally inside the *if* branch. React requires that all Hooks be called

```
1 function Cond() {
2   const [b, setB] = useState(() => false);
3   if (b) {
4     const [s, setS] = useState(() => 0);
5     return s;
6   } else {
7     const h = () => setB(b => !b);
8     return <button onClick={h}> Show
9       </button>; }
```

unconditionally at the top-level of a component, ensuring they execute in the same order on every render. Initially, Cond will show a button to toggle the Boolean state *b* from false to true. When you click “Show” (setting *b* to true), instead of displaying the initial state of *s* which is 0, the UI breaks down at runtime. This stems from React’s implementation detail where each Hook call is identified by its position in a linked list of internal bookkeeping objects.

1.2 The Problem

Hooks have subtle semantics, making it challenging to avoid unexpected behaviors. The rendering process is opaque—it is unclear *how* React registers updates or *when* re-renders occur. Therefore, React developers must follow specific rules or risk breaking their UI (detailed in §3). Worse yet, the host language is oblivious to React’s rules, so it’s up to programmers to adhere to them. Without a clear understanding of the underlying mechanisms, developers must rely on informal resources that typically provide high-level guides rather than explanations of the render semantics, leaving even experienced developers vulnerable to misconceptions about how Hooks actually work.

1.3 Our Solution

We present REACT-TRACE, an operational semantics of React Hooks:

- **REACT-TRACE is a semantics for clarifying the behavior of React Hooks.** REACT-TRACE is *high-level enough* to abstract away implementation details and *precise enough* to aid developers in reasoning about rendering behavior (§4.3). Our semantics helps developers clearly understand confusing UI bugs like infinite re-rendering (§3).

$$\begin{array}{l}
 x, C \in \text{Var} \quad n \in \mathbb{Z} \quad \ell \in \mathbb{N} \quad \text{Prog } P ::= \overline{D} \ e \quad \text{ComDef } D ::= \text{let } C(x) = e \\
 \text{Exp } e ::= () \mid \text{true} \mid \text{false} \mid n \mid x \mid C \mid e \oplus e \mid [\overline{e}] \mid \text{print } e \mid \text{if } e \text{ then } e \text{ else } e \mid e; e \\
 \mid \text{fun } x \rightarrow e \mid e \ e \mid \text{let } x = e \text{ in } e \mid \text{let } (x, x_{\text{set}}) = \text{useState}^\ell e \text{ in } e \mid \text{useEffect } e
 \end{array}$$

Fig. 1. Syntax of REACT-TRACE.

- **REACT-TRACE is a foundation for building semantic-based tools.** REACT-TRACE provides a semantics for React Hooks (§4), which is the first step in designing any semantic-based tool, such as an abstract interpreter [Cousot and Cousot 1977; Rival and Yi 2020]. As an example for a semantic-based tool based on REACT-TRACE, we present an interactive tool that explains and visualizes the behavior of React programs (§5).
- **REACT-TRACE captures the essence of React.** We prove that REACT-TRACE conforms to the key properties of React according to the documentation, as well as presenting an empirical conformance test suite (§6).

Organization. We introduce **useState** and **useEffect** and describe challenges in understanding them in §2, and illustrate common pitfalls from their interactions in §3. We formalize React Hook semantics in §4, present a REACT-TRACE-definitional interpreter and visualizer in §5, and demonstrate conformance with React in §6. We discuss extensions and implications in §7, compare with related work in §8, and conclude with future work in §9.

2 Peculiarities of React Hooks

In this section, we examine the peculiarities of the **useState** (§2.1) and **useEffect** (§2.2) Hooks.

We introduce a simple applicative language for REACT-TRACE that captures the essence of React Hooks and use it throughout the paper to illustrate the examples and the semantics. Thus we decouple ourselves from the subtleties of the complex semantics of JS [Ryu and Park 2024], which is beyond the scope of our research.

Syntax. The syntax of REACT-TRACE is given in Fig. 1. A React program P is a sequence of *component definitions* D , followed by the *main expression* e . A component definition $D = \text{let } C(x) = e$ defines a component named C that accepts parameter x bound in body e . A component name C is capitalized to distinguish it from other identifiers. Expressions e are mostly standard, except for the array view $[\overline{e}]$ for nested views and the Hooks. The **useState** and **useEffect** Hooks can be used only at the top level of the component bodies; they cannot be used as a sub-expression of other expressions—this is enforced by the Rules of React [Meta Platforms, Inc. 2025g]. We check this syntactically during parsing in the REACT-TRACE interpreter (§5). Each **useState** Hook is labeled uniquely with a natural number ℓ . A view represents the structure of the UI that evaluates into either a constant, a closure, or an array view. An array view can represent a nested structure, modeling the JSX syntax [Meta Platforms, Inc. 2022]. A closure used as a view represents an event handler, e.g., a button or a text input. Metavariable $\oplus$ stands for total binary operations over integers. $\text{print } e$ prints to the console. We only consider programs P whose main expressions evaluate to a view.

We model only the **useState** and **useEffect** Hooks in our language, as they are the most essential Hooks that are closely coupled with the rendering of components.

- The **useState** Hook provides a way to manage state in components, making it a cornerstone for defining user interactions in components. **useState** returns a pair of values: the current state x and a setter function x_{set} that updates the state.

- The **useEffect** Hook enables running arbitrary code e after a component is rendered. This Hook makes it useful to synchronize with the “outside world,” but can lead to complex behavior when it triggers a re-render. The expression passed to **useEffect**, e , is called an *Effect* [Meta Platforms, Inc. 2025i].

Actually, as we will see in §3, the common pitfalls of Hooks are caused by misusing **useState** and **useEffect** in combination. We now explain how these Hooks are used in React components in §§2.1 and 2.2, before presenting their exact semantics in §4.

2.1 The Peculiarity of useState

Among the top 45 most frequent StackOverflow questions with the tag [react-hooks], three—including the most frequent question by Pranjal [2019]—are about the semantics and timing of the **useState** Hook and its state updates [Pranjal 2019; Tang 2019; vadirn 2018].

1 **let** Counter $x =$ Consider a counter that increments its value by two when clicked. If our
 2 **let** $s := 0$ **in** language had mutable variables, one might naïvely write the code on the
 3 $[s, \text{button} (\text{fun } _ \rightarrow$ left by defining a mutable variable s and adding two to it when the button
 4 $s := s+2)];$; is clicked. However, this approach presents two significant problems:
 5 Counter 0

- (1) The variable s would be rebound to 0 every time the component is invoked, preventing the runtime from re-reading the component without overwriting the previous state.
- (2) Even if the previous problem were somehow solved, to determine when to update the UI to synchronize with the states, the runtime would need to track all mutable variables in each component, which would be inefficient in practice.

The **useState** Hook addresses these issues by providing functionality analogous to mutable variables in imperative languages, yet with a functional approach. Instead of directly modifying state through assignment, the state is managed by the React runtime and can only be updated by the setter function returned by the Hook. The **useState** Hook allows the runtime to be aware of the state and handle its bookkeeping, therefore addressing the aforementioned two problems.

Using **useState**, we can correctly implement a functioning Counter as shown on the right. **useState** returns a pair of values: s that stores the current state and the setter function setS that accepts an updater function. Initially, s holds the value of x . On subsequent renders, React is able to provide the correct current state without reinitializing to x .

When a button is pressed, the callback passed to it calls setS twice with the updater $\text{fun } s \rightarrow s+1$. Unlike direct assignment to s in the previous example, this does not immediately update s , but queues the update [Meta Platforms, Inc. 2025c]. The update is processed by the runtime during the next render, ensuring that the view is re-rendered with the updated state. This raises a question:

Precisely when in the runtime is this update handled?

To investigate when the callback provided to setS is executed, we can add diagnostic prints:

<pre> 1 let Counter $x =$ 2 print "Counter"; 3 let (s, setS) = useState x in 4 print "Return"; 5 $[s, \text{button} (\text{fun } _ \rightarrow$ 6 $\text{setS} (\text{fun } s \rightarrow s+1);$ 7 $\text{setS} (\text{fun } s \rightarrow \text{print "Update"; } s+1))];;$ 8 Counter 0 </pre>	Console <pre> 1 Counter 2 Return button 3 Counter 4 Update 5 Return </pre>
---	--

The console output shows that Counter and Return are printed during the initial rendering. When the button is clicked, Update is printed *after* Counter and *before* Return—a behavior that is not immediately apparent from the code alone.

In fact, the official React documentation does not specify exactly when the queued updates are processed, other than that they are processed sometime during the next render. We will clearly define how this happened by providing a formal semantics of REACT-TRACE in §4.

2.2 The Peculiarity of useEffect

While **useState** hooks into the React runtime to manage component state, **useEffect** hooks into the rendering lifecycle of components. Every time a component is rendered, an Effect—a suspended computation or a thunk—provided to the **useEffect** Hook is executed. Effect allows developers to run arbitrary logic after the render. Note that in React, it is possible to specify a set of variables called *dependencies*, so that the Effect runs only if those variables have changed. We only consider the simplest form of **useEffect** in our language, where the Effect is run unconditionally after each render, and it is a straightforward extension to support dependencies.

```

1 let Counter x =
2   let (s, setS) = useState x in
3   useEffect (print (if s mod 2 = 0
4     then "Even" else "Odd"));
5   [s, button (fun _ ->
6     setS (fun s -> s+1);
7     setS (fun s -> s+1))]);
8 Counter 0

```

Building upon the previous Counter example, we can log Even or Odd depending on the value of the counter. The **useEffect** Hook makes this straightforward, as shown on the left. This implementation prints either Even or Odd to the console after each render based on the parity of the counter. This “delayed” logging is possible as an Effect expression passed to **useEffect** is unevaluated¹ until after the component has rendered.

While a simple print as an Effect may seem trivial, this amounts to synchronizing with the outside world, which is the very purpose of the **useEffect** Hook. In real life, this can be a call to some logging system or user analytics system.

To understand a more peculiar aspect of **useEffect**, consider the following SelfCounter where an Effect creates an autonomous rendering cycle:

<pre> 1 let SelfCounter x = 2 let (s, setS) = useState x in 3 print s; 4 useEffect (5 print "Effect"; 6 if s < 3 then 7 setS (fun s -> s + 1)); 8 print "Return"; 9 [s]; 10 SelfCounter 0 </pre>	<table border="0"> <tr> <td></td> <td style="text-align: center;">Console</td> </tr> <tr><td>1</td><td>0</td></tr> <tr><td>2</td><td>Return</td></tr> <tr><td>3</td><td>Effect</td></tr> <tr><td>4</td><td>1</td></tr> <tr><td>5</td><td>Return</td></tr> <tr><td>6</td><td>Effect</td></tr> <tr><td>7</td><td>2</td></tr> <tr><td>8</td><td>Return</td></tr> <tr><td>9</td><td>Effect</td></tr> <tr><td>10</td><td>3</td></tr> <tr><td>11</td><td>Return</td></tr> <tr><td>12</td><td>Effect</td></tr> </table>		Console	1	0	2	Return	3	Effect	4	1	5	Return	6	Effect	7	2	8	Return	9	Effect	10	3	11	Return	12	Effect
	Console																										
1	0																										
2	Return																										
3	Effect																										
4	1																										
5	Return																										
6	Effect																										
7	2																										
8	Return																										
9	Effect																										
10	3																										
11	Return																										
12	Effect																										

This example demonstrates that Effects can create render cycles *without any user interaction*. Initially, the value of s, 0 is printed, followed by Return and Effect messages, showing that the Effect runs after the initial render. After the Effect runs, it updates the state when $s < 3$, triggering a new render automatically. The component autonomously increments from 0 to 3 without any user interaction. When s reaches 3, the Effect still runs (as shown by the final Effect output), but no further updates are queued.

This pattern creates a dangerous pitfall:

Developers can unwittingly trigger excess render cycles with seemingly innocent Effects.

¹In React/JS, an Effect needs to be wrapped inside a callback.

Our formal semantics in §4 captures these patterns, helping developers reason about when and how these cycles occur in their programs. We explore how this pitfall leads to bugs in §§3.1.1 and 3.2.

3 Re-Rendering Pitfalls with React Hooks

Having examined the peculiarities of `useState` and `useEffect` in §2, we now demonstrate how their interaction can lead to common render-related bugs. These pitfalls are challenging as they stem from the opaque render semantics. These issues motivate our formal semantics (§4).

3.1 Infinite Re-Rendering

Infinite re-rendering is perhaps the most catastrophic bug one can encounter when using Hooks. There are two different problems in this category of bugs: an infinite render loop due to always setting a different state in a `useEffect` (§3.1.1) and an infinite re-evaluation of a component body due to a top-level call to a setter function (§3.1.2).

3.1.1 Infinite Render Loop with an Effect. Using `useEffect` can easily lead to an infinite render loop—searching StackOverflow with the query “useEffect” “infinite” returns more than 1600 results [Lee 2025]. We describe the essence of the issue here.

Infinite render loop occurs because setting state within an Effect triggers the runtime to **Check**² if the state has changed, which is implemented by invoking the component body again. If the state whose setter function is called is actually modified, the component re-renders and decides to run the **Effect** again. Essentially, this **Check-Effect** decision cycle creates a render loop. In the following, we show a basic example that renders infinitely many times due to this mechanism.

```
1 let Inf x =
2   let (s, setS) = useState 0 in
3   useEffect (setS (fun s -> s+1));
4   s;;
5 Inf 0
```

The component `Inf` on the left is a simple example that increments the state by one after each render in an Effect. Upon each call to `setS` while executing an Effect, the component “remembers” that its body needs to be scanned again to see if the state has actually changed. We say that the component decides to **Check** in the next render pass. Since `s` always increments, the scan results in the runtime re-rendering `Inf` indefinitely.

This render loop is similar to the example `SelfCounter` shown in §2.2, although this time, the condition that breaks the loop is absent. A generalized issue is presented in §3.2.

3.1.2 Top-Level Call to a Setter Function. Infinite loop caused by a top-level call to a setter function is also a common source of confusion—the most frequent StackOverflow question with both the `[reactjs]` and `[infinite-loop]` tag is about this issue [Tehila 2022].

When a component sets a state in the top-level, the runtime immediately **Checks** the component again by re-evaluating it. This causes React to discard the returned view and re-read the component.

An active **Check** decision during reading a component is treated specially as a signal to *retry* the component before the render, unlike a **Check** decision while running an Effect. Therefore, an unconditional top-level call to a setter function causes an infinite loop and is never correct.

Unlike the infinite render loop bug previously discussed in §3.1.1, `Inf2` on the right does not even reach the screen. `Inf2` simply causes the UI to show a blank screen. Even if the state is always set to the same value `0`, `Inf2` falls into an infinite loop.

```
1 let Inf2 x =
2   let (s, setS) = useState x in
3   setS (fun s -> s);
4   s;;
5 Inf2 0
```

3.2 Unnecessary Re-Rendering

A simple example of an unnecessary re-render triggered by the `useEffect` Hook is demonstrated in the following component `Flicker`. `Flicker` starts with an initial state `0`, and immediately after

²We use **red boldfaced sans-serif** to emphasize a decision, whose semantic meaning is formally introduced in §4.1.


```

1 let Flicker x =
2   let (s, setS) = useState x in
3   useEffect (setS (fun _ -> 42));
4   s;;
5 Flicker 0
    
```

the render, it updates the state to 42, triggering the runtime to **Check** the component and causes a re-render. A swift user might even see the transient state 0 and notice a “flicker” in the UI.

While technically a similar problem to the one discussed in §3.1.1, this is a performance issue, in contrast to the other which was an evident bug. When the component needs to access an external resource in order to set a state, setting a state after the call to **useEffect** may be necessary, but in other scenarios, it is a waste of a render cycle to do so.

Note that this issue can happen inter-component as well in a more subtle manner. In the example on the right, the parent component Parent decides to render the child component Child based on the state *s*. Initially, Parent renders Child, passing its setter function *setS* to Child. After the render, Effect in Child gets invoked, queuing an update to the state *s* of Parent to false. In the end, Parent re-renders, but this time, it will not render Child, as the state *s* becomes false.

```

1 let Child setS =
2   useEffect (setS (fun _ -> false));
3   ();
4 let Parent b =
5   let (s, setS) = useState b in
6   if s then Child setS else ();
7 Parent true
    
```

4 An Operational Semantics for the Essence of React Hooks

We now provide a formal operational semantics REACT-TRACE that is both high-level enough to abstract away React’s implementation details, yet precise enough to reason about rendering behavior (§6). Up until now, we have implicitly discussed “values” using the syntax of REACT-TRACE, and we formally introduce semantic objects in §4.1. Then we describe the operational semantics of REACT-TRACE in §4.2.

There are two layers that constitute the syntax and semantics of React Hooks:

Base layer describes standard computations within components. An excerpt from Fig. 1:

$$\text{Exp } e ::= () \mid \text{true} \mid \text{false} \mid n \mid x \mid e \oplus e \mid \text{print } e \mid \text{if } e \text{ then } e \text{ else } e \mid e; e \mid \text{fun } x \rightarrow e \mid e e \mid \text{let } x = e \text{ in } e \mid \dots$$

Render layer controls the rendering of components. An excerpt from Fig. 1:

$$\text{Exp } e ::= \dots \mid C \mid [\bar{e}] \mid \text{let } (x, x_{\text{set}}) = \text{useState}^{\ell} e \text{ in } e \mid \text{useEffect } e$$

The base logic layer is crucial in order to build a rich UI with complex business logic. For instance, Booleans and conditionals are required to write a toggleable button; Functions and function applications are necessary to perform actions on behalf of user interaction.

Our choice of the base layer is not the only possible one. We have chosen a minimal combination of features that can support the essence of Hooks—we can extend our base with strings, objects, recursive functions, etc. Here, we will not burden ourselves with a layer not too relevant to the render logic. In fact, all these features are included in our implementation of REACT-TRACE (§5).

The render logic layer seems quite minimal, but we have seen its subtleties in §§2 and 3. We include all the base values to be used to represent views, and in addition, we include an array to model nested views and multiple children. This is not a general purpose array—its elimination is when being used at the returning position of a component. An example of an array view has been shown in the running examples in §2. Each **useState** Hook is (implicitly) labeled so that it is uniquely identified. The returned value of **useState** is a pair of current value *x* and the setter function *x_{set}*. **useEffect** Hooks need not be labeled.

Note that we are only concerned with the timing aspects of the render semantics, not the visual layout; hence interactive UI elements such as a button is identified with its event handler. As such, **button** *f* is simply desugared into its event handler *f*.

4.1 Semantic Objects

Semantic objects include base values (§4.1.1) as well as the machinery that models the React runtime and Hooks (§4.1.2). The complete set of semantic objects is reproduced in §A.1 for reference.

4.1.1 Base Values. The semantic objects for base values are formalized as follows:

$$\begin{aligned} \text{Val } v &::= k \mid cl \mid \dots & \text{Clos } cl &::= \langle \lambda x. e, \sigma \rangle \\ \text{Const } k &::= \langle \rangle \mid \mathbf{tt} \mid \mathbf{ff} \mid n & \text{Env } \sigma &::= [\overline{x \mapsto v}] \\ n &\in \mathbb{Z} & \text{Buffer } \omega &::= [\overline{v}] \end{aligned}$$

Base values are standard—constants k include the unit value $\langle \rangle$, Booleans $\mathbf{tt}$ and $\mathbf{ff}$, and integers n . To model the base layer using big-step semantics [Kahn 1987] in §4.2.3, bound variables are recorded in an environment $\sigma = [\overline{x \mapsto v}]$. A function evaluates into a closure $cl = \langle \lambda x. e, \sigma \rangle$. A buffer $\omega = [\overline{v}]$ holds a list of printed values.

4.1.2 React Machinery.

Extended Values. To model the React machinery, we extend base values Val:

$$\begin{aligned} \text{Val } v &::= \dots \mid C \mid cs \mid [\overline{s}] \mid \langle \ell, p \rangle & \text{DefTable } \delta &::= [\overline{C \mapsto \lambda x. e}] \\ \text{ComSpec } cs &::= \langle C, v \rangle & \ell &\in \mathbb{N} \\ \text{ViewSpec } s &::= k \mid cl \mid cs \mid [\overline{s}] & p &\in \text{Path} = \mathbb{N} \end{aligned}$$

A component name C is a value as-is. All component names are determined statically and their definitions are looked up using a *definition table* $\delta = [\overline{C \mapsto \lambda x. e}]$. Thus (mutually) recursive components are permitted in REACT-TRACE, and we do not model legacy higher-order components [Meta Platforms, Inc. 2025b]. A *view spec* is either a constant value k , a closure cl , a component spec cs , or an array view spec $[\overline{s}]$. A constant and a closure view spec represent leaf views, whereas a component and an array view spec represent composite views. A constant view spec is simply realized into a view that shows its string representation. A closure view spec, when realized into a view, represents an event handler—this models UI elements such as a button, an input field, a checkbox, etc. A component spec $cs = \langle C, v \rangle$ is simply a pair of a component name and its argument. This represents a specification of a view that may be realized into a view hierarchy. An array view spec $[\overline{s}]$ is an array of view specs s , representing structured child views.

A setter closure $\langle \ell, p \rangle$, which is the semantic value of a setter function, is a pair of a label ℓ from the originating **useState** Hook and a *path* p to the corresponding view in the view tree. A path p is a unique natural number used to identify each view in the view tree. We shall take a look at the view tree structure m in a moment.

Tree Memory. We introduce *tree memory* m to model the view hierarchy:

$$\begin{aligned} \text{TreeMem } m &::= [\overline{p \mapsto \pi}] \\ \text{View } \pi &::= \{\text{spec: } cs, \text{dec: } \{\overline{d}\}, \text{sttst: } \rho, \text{effq: } q, \text{child: } t\} \\ \text{Decision } d &::= \text{Check} \mid \text{Effect} \\ \text{Tree } t &::= k \mid cl \mid p \mid [\overline{t}] & \text{JobQ } q &::= [\overline{cl}] \\ \text{SttStore } \rho &::= [\overline{\ell \mapsto \{\text{val: } v, \text{sttq: } q\}}] & \text{Context } \Sigma &::= m \mid \pi \end{aligned}$$

The entire view hierarchy is stored in a tree memory m which maps each path p to a view π . A view is a realized component that is mounted or to-be-mounted into a tree memory. Each mounted view π is assigned a unique path p where p is a valid path of a tree memory m , i.e., $p \in \text{dom } m$. It consists of a component spec cs , a decision d , an effect queue q , and a child tree t . When a component body is evaluated, the runtime keeps track of its decision set—**Check** for checking the component for retry or re-render, and **Effect** for running Effects after this render. Decisions **Check**

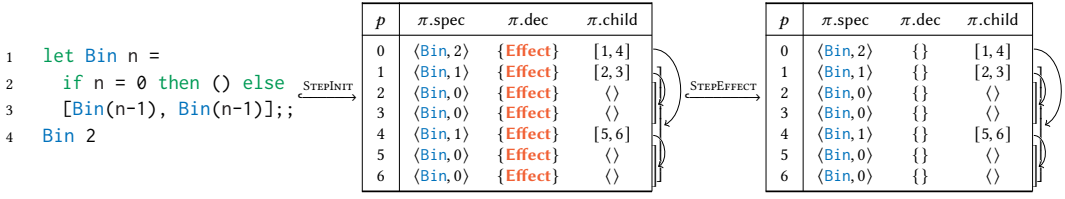


Fig. 2. Visualization of a tree memory and render step transitions for a simple recursive component.

and **Effect** have been explained informally in §§3.1.1 and 3.1.2, and all components first begin with an empty decision set. A state store ρ for each view is used to store the current state value v and the queued updates q from setter function from the **useState** Hook. These are keyed using the corresponding label ℓ of **useState**. An effect queue q is a queue of Effects from the **useEffect** Hook. A child t is either a terminal view k , an event handler cl , a path p to another view, or an array of children $[t]$. We also introduce a context Σ , which can be either a tree memory m or a view π . This distinction exists because one evaluation rule (APPSETNORMAL) requires the whole tree memory m , while the others only require the local view π in their context. The whole-memory context is used to describe a component updating another, e.g., a button in a child component updating its parent's state.

To give a visual intuition of how tree memory represents view hierarchy, Fig. 2 shows a tree memory for a simple recursive component `Bin` that creates a complete binary tree of height 2. The root tree is assigned path $p = 0$, and each subtree receives a unique path in the order of initialization (Fig. 7, §4.2.3). The view hierarchy can be reconstructed by traversing the child field of each view in the tree memory. The render step transitions **STEPINIT** and **STEPEFFECT** along with the decisions shown in Fig. 2 are explained in detail in §4.2.1.

Phase. While evaluating an expression, we maintain a phase ϕ :

$$\text{Phase } \phi ::= \text{Init} \mid \text{Succ} \mid \text{Normal}$$

Init³ phase is used when evaluating a component for the first time to be mounted into a tree memory. At this phase, all states are evaluated from the initial expressions of **useStates**. Successive calls to a component function are done in **Succ** phase, where states are retrieved from a state store. The main expression, Effects, and event handlers are evaluated in **Normal** phase.

Mode. We model each (re-)render as a transition of global states, and we introduce a mode μ to keep track of the state of the REACT-TRACE engine:

$$\text{Mode } \mu ::= \text{⊗ (rendered)} \mid \cup (\text{check}) \mid \bullet (\text{event loop})$$

Rendered mode $\otimes$ represents a state where the UI has been rendered on the screen and is waiting for the queued Effects to run. Check mode $\cup$ represents a state where the runtime will check for a re-render. When a re-render is not required, we enter event loop mode $\bullet$ and wait for an input.

4.2 Operational Semantics

All semantic functions⁴ and evaluation relations, along with their dependencies and required contexts, are summarized in Fig. 3. We give a brief overview of the semantic functions and relations before we formally introduce them in §§4.2.3 to 4.2.5:

³We use **blue sans-serif** to emphasize a phase.

⁴Semantic functions are typeset using an *italicized sans-serif* font.

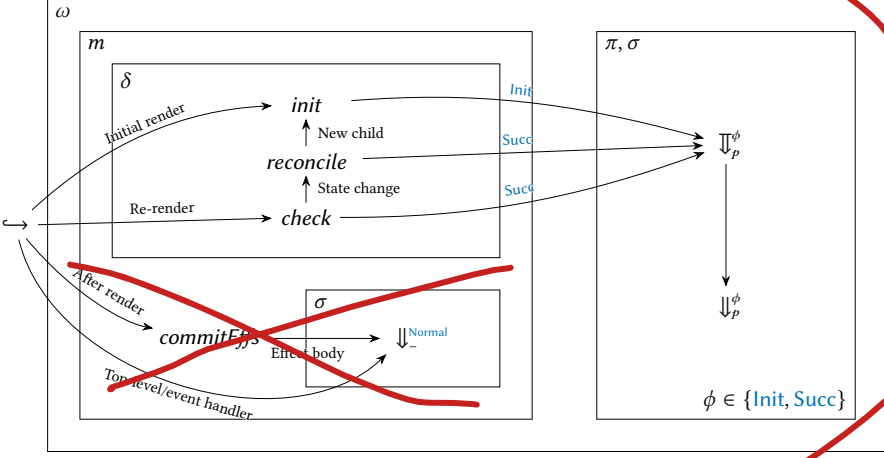


Fig. 3. Semantic function dependencies.

- $\langle e, \delta \rangle$ or $\langle t, m, \omega, \delta, \mu \rangle \hookrightarrow \langle t, m', \omega', \delta, \mu' \rangle$ is a render step transition (Fig. 4). Read this as “Initially, under definition table δ , main expression e transitions to root tree t , tree memory m' , and output buffer ω' , entering mode μ' ,” or “Given root tree t under definition table δ , tree memory m and output buffer ω in mode μ transition to m' and ω' , entering mode μ' .”
- $\Sigma, \sigma \vdash e \Downarrow_p^\phi v, \Sigma', \omega$ is an evaluation of an expression (Fig. 5). Read this as “In phase ϕ at path p , expression e under environment σ and context Σ evaluates to value v , producing a modified context Σ' and output ω .”
- $\pi, \sigma \vdash e \Downarrow_p^\phi s, \pi', \omega$ is a possibly retrying evaluation of a component body until it becomes idle (Fig. 6). Read this as “In phase ϕ at path p , component body e of view π under environment σ eventually evaluates to view spec s , producing a modified view π' and output ω .”
- $m, \delta \vdash \text{init}(s) = \langle t, m', \omega \rangle$ is a rendering of a view spec in an initial render (Fig. 7). Read this as “Under definition table δ , view spec s initially renders into tree t , modifying tree memory from m to m' and printing ω .”
- $m \vdash \text{commitEfs}(t) = \langle m', \omega \rangle$ is committing queued Effects (Fig. 8). Read this as “Committing Effects of tree t modifies tree memory from m to m' and prints ω .”
- $m, \delta \vdash \text{check}(t) = \langle \mu, m', \omega \rangle$ is checking and updating a tree for a re-render (Fig. 9). Read this as “Under definition table δ , tree t re-renders with modified tree memory m' from m and prints ω ” when $\mu = \otimes$, and “Under definition table δ , tree t does not re-render but modifies tree memory from m to m' and prints ω ” when $\mu = \bullet$; μ cannot be $\cup$.
- $m, \delta \vdash \text{reconcile}(t, s) = \langle t', m', \omega \rangle$ is reconciling a tree with a possibly updated view spec (Fig. 10). Read this as “Under definition table δ , tree t is reconciled as t' with respect to view spec s , modifying tree memory from m to m' and printing ω .”

Notation. We explain notation used in the evaluation rules. When a rule has premises J_l for $l \leq i \leq u$, we write $(J_l)_{l=1}^u$. We use $++$ for list concatenation, e.g., $x ++ x'$, and $++_{i=1}^u x_i$ for concatenating $x_1, \dots, x_u$. For bindings X , $X[x \mapsto v]$ updates X with $x \mapsto v$, and $X[x]$ retrieves the value v when X has a binding $x \mapsto v$. For a structure X with a field f , $X\{f: v\}$ updates field f to v , and $X.f$ accesses field f . We use meta-level let-bindings “let $x = v$ in $\dots$ ” inside meta-level structure or list updates when convenient. Meta-level conditional $(b \text{ ? } x \text{ : } y)$ chooses x when b holds and chooses y otherwise.

4.2.1 Render Loop. We model the React event loop with render step transitions in Fig. 4.

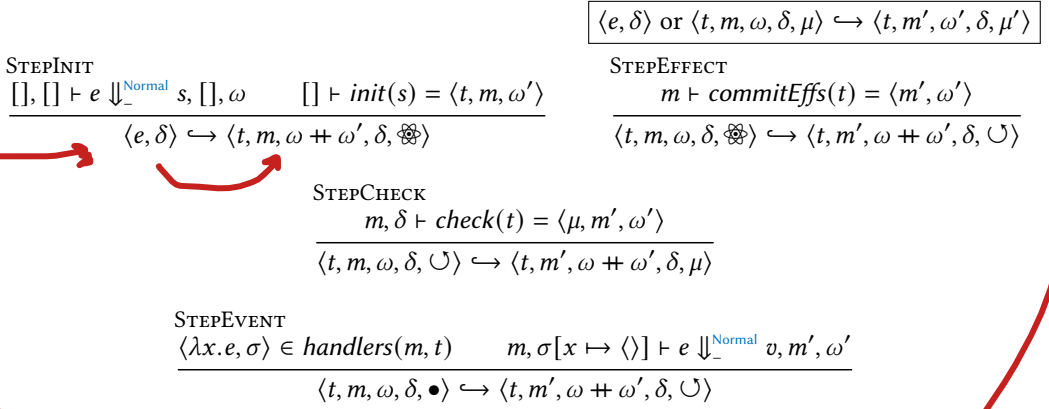


Fig. 4. Render step transitions.

Given a program P , the loop begins with an initial state $\langle e, \delta \rangle$, where e is the main expression of P and δ is the definition table of all components defined in P . Note that the definition table δ and the root tree t (once initialized) remain unchanged during transitions.

First, we evaluate the main expression e into a view spec s and initialize it into a tree t with an updated tree memory m ; we enter the rendered mode $\otimes$ (STEPINIT). We call this a rendered mode because the root tree t and all of its descendants as described in m have been *rendered on screen*. Then we commit all the queued Effects of t and its descendants, after which we enter the check mode $\cup$ (STEPEFFECT). In check mode $\cup$, we check for any state update under t . If there is an update, we render again and enter the rendered mode $\otimes$; otherwise, we enter the event loop mode $\bullet$ (STEPCHECK). In event loop mode $\bullet$, we wait for user input to be handled by an event handler. If input occurs, we evaluate the corresponding handler body, which then requires a state update check (STEPEVENT).

Returning to the simple example shown in Fig. 2, all the views in the tree memory contain **Effect** decisions after the STEPINIT transition, as they are freshly mounted and React needs to check for any defined Effects (of which there are none in the example). All the **Effect** decisions are then cleared after the STEPEFFECT transition. No state updates have been made—there are no **Check** decisions—and STEPCHECK transitions to the same tree memory, this time in event loop mode $\bullet$.

To elaborate on how the transition STEPINIT evaluates the main expression and initializes the evaluated view spec, we turn our attention to the evaluation rules for expressions (§4.2.2) and the initialization rules (§4.2.3).

4.2.2 Evaluating an Expression.

Single Evaluation. We present how an expression is evaluated in a big-step semantics in Fig. 5. The rules relevant to the render logic are included here, which includes **useState** and the **useEffect** Hooks, component application, and state updates. Standard rules related to the base logic are not presented—APPFUNC and PRINT are included for clarity. The complete operational semantics of expressions is available in §A.2.

An environment σ , context Σ , phase ϕ , and path p are required for evaluating an expression e . A path is not available (written as $-$ in Fig. 5) in the **Normal** phase. Evaluation rules that are only applicable in the context of a component body evaluation (APPSETCOMP, STTBIND, STTREBIND, and EFF), i.e., in the **Init** and **Succ** phase, require the context Σ to be a view π . A noteworthy case is

$$\boxed{\Sigma, \sigma \vdash e \Downarrow_p^\phi v, \Sigma', \omega}$$

APPFUNC

$$\frac{\Sigma, \sigma \vdash e_1 \Downarrow_p^\phi \langle \lambda x. e, \sigma' \rangle, \Sigma_1, \omega_1 \quad \Sigma_1, \sigma \vdash e_2 \Downarrow_p^\phi v_2, \Sigma_2, \omega_2 \quad \Sigma_2, \sigma' [x \mapsto v_2] \vdash e \Downarrow_p^\phi v', \Sigma', \omega'}{\Sigma, \sigma \vdash e_1 e_2 \Downarrow_p^\phi v', \Sigma', \omega_1 \uplus \omega_2 \uplus \omega'}$$

APPCOM

$$\frac{\Sigma, \sigma \vdash e_1 \Downarrow_p^\phi C, \Sigma_1, \omega_1 \quad \Sigma_1, \sigma \vdash e_2 \Downarrow_p^\phi v, \Sigma_2, \omega_2}{\Sigma, \sigma \vdash e_1 e_2 \Downarrow_p^\phi \langle C, v \rangle, \Sigma_2, \omega_1 \uplus \omega_2}$$

APPSETCOMP

$$\frac{\pi, \sigma \vdash e_1 \Downarrow_p^\phi \langle \ell, p \rangle, \pi_1, \omega_1 \quad \pi_1, \sigma \vdash e_2 \Downarrow_p^\phi cl, \pi_2, \omega_2 \quad \phi \in \{\text{Init}, \text{Succ}\}}{\pi, \sigma \vdash e_1 e_2 \Downarrow_p^\phi \langle \rangle, \pi_2 \left\{ \begin{array}{l} \text{let } \rho = \pi_2.\text{sttst}, vq = \rho[\ell] \text{ in} \\ \text{dec: } \pi_2.\text{dec} \cup \{\text{Check}\} \\ \text{sttst: } \rho[\ell \mapsto vq\{\text{sttq: } vq.\text{sttq} \uplus [cl]\}] \end{array} \right\}, \omega_1 \uplus \omega_2}$$

APPSETNORMAL

$$\frac{m, \sigma \vdash e_1 \Downarrow_{-}^{\text{Normal}} \langle \ell, p \rangle, m_1, \omega_1 \quad m_1, \sigma \vdash e_2 \Downarrow_{-}^{\text{Normal}} cl, m_2, \omega_2}{m, \sigma \vdash e_1 e_2 \Downarrow_{-}^{\text{Normal}} \langle \rangle, m_2 \left\{ \begin{array}{l} \text{let } \pi = m_2[p], \rho = \pi.\text{sttst}, vq = \rho[\ell] \text{ in} \\ \text{dec: } \pi.\text{dec} \cup \{\text{Check}\} \\ p \mapsto \pi \left\{ \begin{array}{l} \text{dec: } \pi.\text{dec} \cup \{\text{Check}\} \\ \text{sttst: } \rho[\ell \mapsto vq\{\text{sttq: } vq.\text{sttq} \uplus [cl]\}] \end{array} \right\} \end{array} \right\}, \omega_1 \uplus \omega_2}$$

STTBIND

$$\frac{\pi, \sigma \vdash e_1 \Downarrow_p^{\text{Init}} v_1, \pi_1, \omega_1 \quad \pi_1 \left\{ \begin{array}{l} \text{sttst: } \pi_1.\text{sttst} \left[\ell \mapsto \left\{ \begin{array}{l} \text{val: } v_1 \\ \text{sttq: } [] \end{array} \right\} \right] \end{array} \right\}, \sigma \left[\begin{array}{l} x \mapsto v_1 \\ x_{\text{set}} \mapsto \langle \ell, p \rangle \end{array} \right] \vdash e_2 \Downarrow_p^{\text{Init}} v_2, \pi_2, \omega_2}{\pi, \sigma \vdash \text{let } (x, x_{\text{set}}) = \text{useState}^\ell e_1 \text{ in } e_2 \Downarrow_p^{\text{Init}} v_2, \pi_2, \omega_1 \uplus \omega_2}$$

STTREBIND

$$\frac{\pi_0.\text{sttst}[\ell] = \{\text{val: } v_0, \text{sttq: } [\overline{\langle \lambda x_i. e'_i, \sigma_i \rangle}_{i=1}^n]\} \quad (\pi_{i-1}, \sigma_i [x_i \mapsto v_{i-1}] \vdash e'_i \Downarrow_p^\phi v_i, \pi_i, \omega_i)_{i=1}^n \quad \pi_n \left\{ \begin{array}{l} \text{dec: } \pi_n.\text{dec} \cup \{v_n \neq v_0 \ ? \ \{\text{Effect}\} \ \& \ \{\}\} \} \\ \text{sttst: } \pi_n.\text{sttst}[\ell \mapsto \{\text{val: } v_n, \text{sttq: } []\}] \end{array} \right\}, \sigma \left[\begin{array}{l} x \mapsto v_n \\ x_{\text{set}} \mapsto \langle \ell, p \rangle \end{array} \right] \vdash e_2 \Downarrow_p^\phi v, \pi', \omega'}{\pi_0, \sigma \vdash \text{let } (x, x_{\text{set}}) = \text{useState}^\ell e_1 \text{ in } e_2 \Downarrow_p^{\text{Succ}} v, \pi', (\uplus_{i=0}^n \omega_i) \uplus \omega'}$$

combined
output
buffer

EFF

$$\frac{\phi \in \{\text{Init}, \text{Succ}\}}{\pi, \sigma \vdash \text{useEffect } e \Downarrow_p^\phi \langle \rangle, \pi \{\text{effq: } \pi.\text{effq} \uplus [\langle \lambda _ . e, \sigma \rangle]\}, []}$$

PRINT

$$\frac{\Sigma, \sigma \vdash e \Downarrow_p^\phi v, \Sigma', \omega}{\Sigma, \sigma \vdash \text{print } e \Downarrow_p^\phi \langle \rangle, \Sigma', \omega \uplus [v]}$$

Fig. 5. Evaluation of an expression (an excerpt).

APPSETNORMAL, the only rule applicable in the **Normal** phase only. All other rules permit both π and m as a context Σ . The output buffer ω stores the printed values sequentially (PRINT).

It is important to note that component definitions are not required during evaluation of an expression. Hence, a component application is nothing more than packing the evaluated component name C and the argument v in a pair (APPCom). The actual function bound to the component name is neither invoked nor required at this point.

$$\begin{array}{c}
 \boxed{\pi, \sigma \vdash e \Downarrow_p^\phi s, \pi', \omega} \\
 \\
 \text{EVALONCE} \\
 \frac{\pi\{\text{dec}: \pi.\text{dec} \setminus \{\text{Check}\}, \text{effq}: []\}, \sigma \vdash e \Downarrow_p^\phi s, \pi', \omega \quad \text{Check} \notin \pi'.\text{dec} \quad \phi \in \{\text{Init}, \text{Succ}\}}{\pi, \sigma \vdash e \Downarrow_p^\phi s, \pi', \omega} \\
 \\
 \text{EVALMULT} \\
 \frac{\pi\{\text{dec}: \pi.\text{dec} \setminus \{\text{Check}\}, \text{effq}: []\}, \sigma \vdash e \Downarrow_p^\phi s, \pi', \omega \quad \text{Check} \in \pi'.\text{dec} \quad \pi', \sigma \vdash e \Downarrow_p^{\text{Succ}} s', \pi'', \omega' \quad \phi \in \{\text{Init}, \text{Succ}\}}{\pi, \sigma \vdash e \Downarrow_p^\phi s', \pi'', \omega \uparrow \omega'}
 \end{array}$$

Fig. 6. Retrying evaluation of a component body.

The **useState** Hook behaves differently in the **Init** and **Succ** phases. In the **Init** phase, the initial value v_1 is evaluated from the argument e_1 , which is then stored in the view's state store `sttst` at label ℓ of **useState**, along with an empty state update queue `sttq` (`STTBIND`). In addition, the state variable x and the setter function x_{set} are bound to the initial value v_1 and the setter closure, which is a pair $\langle \ell, p \rangle$, so that the value of the state can be read and set.

In the **Succ** phase, all the queued updates in `sttq` are processed while evaluating the corresponding **useState** Hook (`STTBIND`). After evaluating all the updates, we compare the final state value v_n with the initial value v_0 . If they are equivalent ($v_n \equiv v_0$), we simply keep the previous decision `dec`. Otherwise ($v_n \neq v_0$), the component adds the **Effect** decision to its decision so that it runs its Effects after rendering. The state update queue `sttq` is flushed and the state variable and the setter function are bound appropriately.

State updates via setter function applications behave differently when evaluating a component ($\phi \in \{\text{Init}, \text{Succ}\}$) compared to when evaluating Effects or event handlers ($\phi = \text{Normal}$). During component body evaluation, calling a setter function of another component is not allowed, hence the path p in the setter closure must match with the context (`APPSETCOMP`). The official React runtime logs an error message when a component evaluation invokes a setter function of another component. Setting the component's state during the evaluation of its body adds the **Check** decision and queues the provided update closure cl . The **Check** decision triggers a re-evaluation of the component—this behavior is formalized in Fig. 6.

State updates in the **Normal** phase lift the restriction of “inter-component” updates. The setter closure $\langle \ell, p \rangle$ is used to queue the update closure cl in the correct tree at path p and the correct label ℓ in the state queue `sttq` (`APPSETNORMAL`). The **Check** decision is added to the view's decision, to mark that a state update has been queued. The marked views are later processed in batches (§4.2.5).

Retrying Evaluation. The evaluation of a component body starts with a *retrying evaluation*, where the body is repeatedly evaluated until the **Check** decision is no longer present. Before each evaluation, the Effect queue is cleared so that Effects are re-collected during execution. The rationale is that re-evaluated view specs are discarded except for the last one, and Effects associated with the discarded view specs must also be discarded. In addition, the **Check** decision is removed beforehand to determine whether the evaluation triggers another **Check**. The component body is then evaluated, and evaluation stops when **Check** is no longer present (`EVALONCE`). If the evaluation results in another **Check** decision, re-evaluation is performed in the **Succ** phase (`EVALMULT`).

Note that a component may indefinitely decide to **Check**, which will lead to an infinite retrieval issue described in §3.1.2. React runtime raises an exception after 25 retrievals.

$$\begin{array}{c}
\boxed{m, \delta \vdash \text{init}(s) = \langle t, m', \omega \rangle} \\
\\
\text{INITCONST} \quad \frac{}{m, \delta \vdash \text{init}(k) = \langle k, m, [] \rangle} \qquad \text{INITCLOS} \quad \frac{}{m, \delta \vdash \text{init}(cl) = \langle cl, m, [] \rangle} \\
\\
\text{INITARRAY} \quad \frac{(m_{i-1}, \delta \vdash \text{init}(s_i) = \langle t_i, m_i, \omega_i \rangle)_{i=1}^n}{m_0, \delta \vdash \text{init}([\overline{s_i}]_{i=1}^n) = \langle [\overline{t_i}]_{i=1}^n, m_n, \text{++}_{i=1}^n \omega_i \rangle} \\
\\
\text{INITCOM} \quad \frac{\begin{array}{c} m \vdash p \text{ fresh} \\ \delta[C] = \lambda x. e \quad \{ \text{spec}: \langle C, v \rangle, \text{dec}: \{ \}, \text{sttst}: [], \text{effq}: [], \text{child}: \langle \rangle \}, [x \mapsto v] \vdash e \mathbb{J}_p^{\text{init}} s, \pi, \omega \\ m[p \mapsto \pi], \delta \vdash \text{init}(s) = \langle t, m', \omega' \rangle \end{array}}{m, \delta \vdash \text{init}(\langle C, v \rangle) = \langle p, m' [p \mapsto \pi \{ \text{dec}: \{ \text{Effect} \}, \text{child}: t \}], \omega \text{ ++ } \omega' \rangle}
\end{array}$$

Fig. 7. Initialization of a view spec.

4.2.3 Initial Render. When a view is initially rendered, it is initialized from a view spec (Fig. 7). A constant (INITCONST), closure (INITCLOS), and array view spec initialization (INITARRAY) are straightforward.

To initialize a component spec $\langle C, v \rangle$, a fresh path p with respect to the tree memory m is generated, and the definition table δ is used to look up the component definition for name C (INITCOM). First, an empty view with the component spec, an empty decision set, and a (placeholder) unit child is created. This view is used as a context to evaluate the component body in the **init** phase with the parameter x bound to v , which produces a view spec s and an updated view π . After mounting the updated π at path p in tree memory m , view spec s is recursively initialized to get the child tree t . Finally, the **Effect** decision is added to the view to ensure the queued Effects are run after the initial render, and the child field is set to t . Note that the initialization of the child tree does not modify the parent view π .

4.2.4 Committing Effects After Render. After a view has been rendered, the queued Effects must be executed. The rules in Fig. 8 describe this process. For a constant (COMMITEFFSCONST) and a closure (COMMITEFFSCLOS), there are no Effects to commit, so the tree memory remains unchanged. For an array of trees (COMMITEFFSARRAY), we recursively commit the Effects of each child tree.

For a path to a view, Effects are committed only when the view has decided to. If the view has not decided to commit **Effects**, only the Effects of its child are executed, and the view's Effects are skipped (COMMITEFFSPATHIDLE). If the view's decision includes **Effect**, the Effects of its child are committed first, and then the view's Effects are executed in order (COMMITEFFSPATH). Each Effect is evaluated in the **Normal** phase, allowing it to update any component's states in the tree.

Effects execution follows a post-order traversal—the child's Effects are executed first, followed by those of its parent. Additionally, Effects are executed following their original syntactic order.

4.2.5 Checking, Re-Render, and Reconciliation.

Checking for Re-Render. In check mode $\cup$, which is after committing Effects or handling an event, the runtime checks and re-renders a tree if needed. This is carried out with semantic function *check* shown in Fig. 9. When a tree or any of its descendants requires a re-render, *check* performs the re-render and returns either of $\otimes$ or $\bullet$. It also returns the modified tree memory.

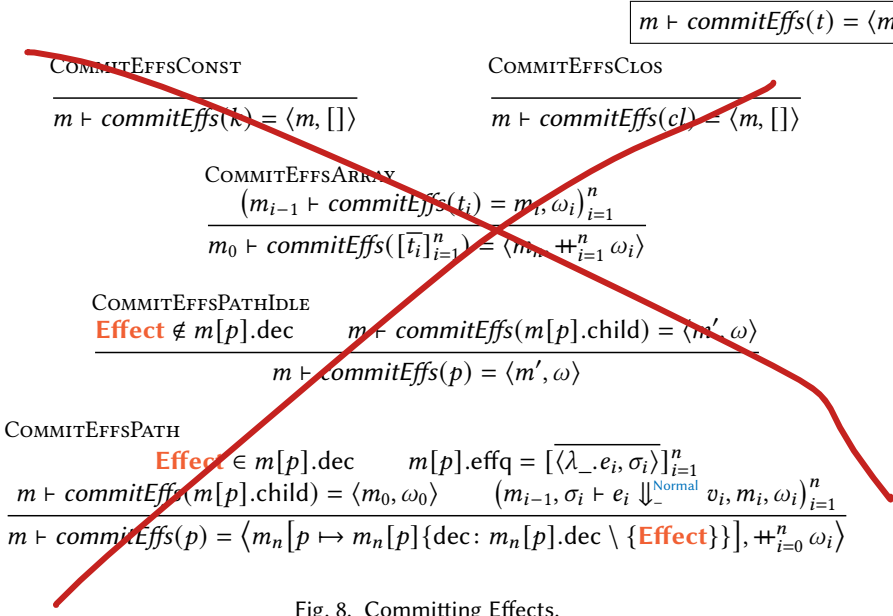


Fig. 8. Committing Effects.

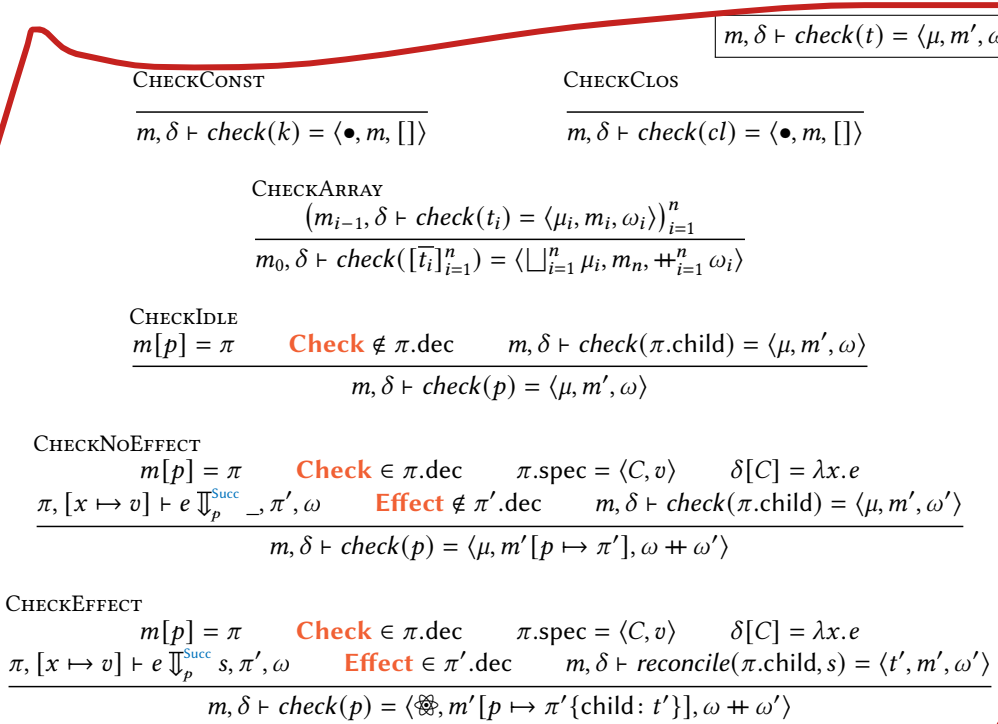


Fig. 9. Checking a tree for re-render.

$$\boxed{m, \delta \vdash \text{reconcile}(t, s) = \langle t', m', \omega \rangle}$$

$$\frac{\text{RECONCILEARRAY} \quad (m_{i-1}, \delta \vdash \text{reconcile}(t_i, s_i) = \langle t'_i, m_i, \omega_i \rangle)_{i=1}^n}{m_0, \delta \vdash \text{reconcile}([\bar{t}_i]_{i=1}^n, [\bar{s}_i]_{i=1}^n) = \langle [\bar{t}'_i]_{i=1}^n, m_n, \text{++}_{i=1}^n \omega_i \rangle}$$

$$\frac{\text{RECONCILECOMEffect} \quad \begin{array}{l} m[p] = \pi \quad \pi.\text{spec} = \langle C, _ \rangle \quad \delta[C] = \lambda x.e \\ \pi\{\text{spec}: \langle C, v \rangle\}, [x \mapsto v] \vdash e \Downarrow_p^{\text{Succ}} s, \pi', \omega \quad m, \delta \vdash \text{reconcile}(\pi.\text{child}, s) = \langle t', m', \omega' \rangle \end{array}}{m, \delta \vdash \text{reconcile}(p, \langle C, v \rangle) = \langle p, m'[p \mapsto \pi' \{\text{dec}: \{\text{Effect}\}, \text{child}: t'\}], \omega \text{++} \omega' \rangle}$$

$$\frac{\text{RECONCILECOMNew} \quad \begin{array}{l} m[p].\text{spec} = \langle C', _ \rangle \quad C \neq C' \quad m, \delta \vdash \text{init}(\langle C, v \rangle) = \langle t', m', \omega \rangle \end{array}}{m, \delta \vdash \text{reconcile}(p, \langle C, v \rangle) = \langle t', m', \omega \rangle}$$

$$\frac{\text{RECONCILEOTHER} \quad \begin{array}{l} \langle t, s \rangle \neq \langle [\bar{t}], [\bar{s}] \rangle \quad \langle t, s \rangle \neq \langle p, \langle C, v \rangle \rangle \quad m, \delta \vdash \text{init}(s) = \langle t', m', \omega \rangle \end{array}}{m, \delta \vdash \text{reconcile}(t, s) = \langle t', m', \omega \rangle}$$

Fig. 10. Reconciliation of a tree with a view spec.

For a constant value (CHECKCONST) and a closure (CHECKCLOS), no render is needed and *check* always returns $\bullet$ without any modification to the tree memory. For a tree array, we check each of its trees recursively in sequence (CHECKARRAY). Note the $\sqcup$ operation in CHECKARRAY, defined as a commutative operator between $\{\bullet, \otimes\}$ where $\bullet \sqcup \bullet = \bullet$ and $\otimes \sqcup \otimes = \otimes \sqcup \bullet = \otimes$. This is because returning $\otimes$ indicates that any descendant has been re-rendered.

The interesting cases occur with path references to views. An idle view that does not need **Checking** simply skips and recurse on its child (CHECKIDLE). If a view's decision includes **Check**, we need to re-evaluate the component body.

When the re-evaluation of a view with **Check** decision results in a decision without **Effect**, it means that the view eventually settled with the same state as before, and no re-render happens (CHECKNOEFFECT). When the re-evaluation decides to commit **Effects**, we need to reconcile the child tree against the new view spec (CHECKEFFECT). In this case, we return $\otimes$ to indicate that the view's state has changed. Note that the re-evaluation premises of CHECKNOEFFECT and CHECKEFFECT do not modify the child $\pi.\text{child}$, as an evaluation does not touch the child field (§4.2.2).

Reconciliation. When some states of a view update, it needs to be *reconciled* with the updated view spec, which is described in Fig. 10. For reconciling an array tree against an array view spec, we reconcile each child tree with the corresponding view spec (RECONCILEARRAY).

For a path to a component view, if the component name is the same as before, we re-evaluate the component body in the **Succ** phase and then reconcile the old child with the new view spec (RECONCILECOMEffect). If the component name has changed, we re-initialize the view spec as this is a completely new component compared to before (RECONCILECOMNew).

For all other cases, such as when a constant, closure, or array tree transitions to a different type, we re-initialize the view spec from scratch (RECONCILEOTHER).

This reconciliation process allows React to efficiently update the UI when state changes occur, preserving existing (virtual) DOM nodes where possible, and only rebuilding the parts that have actually changed [Staff 2016].

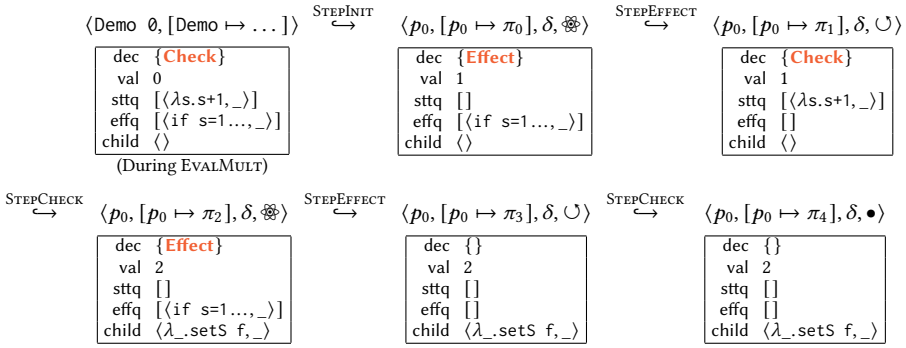
4.3 An Illustrative Example

To illustrate the operational semantics of Hooks in action, we walk through the execution of the Demo component on the right. The example incorporates both the **useState** and **useEffect** Hooks, showcasing the unnecessary re-rendering issue (§3.2) and the top-level setter issue (§3.1.2). Demo also demonstrates reconciliation by updating its child view from `()` to **button** (or a closure) after the re-render.

```

1 let Demo x =
2   let (s, setS) = useState x in
3   let f = fun s -> s + 1 in
4   if s = 0 then setS f;
5   useEffect (if s = 1 then setS f);
6   if s <= 1 then () else
7     button (fun _ -> setS f);;
8 Demo 0
    
```

The render step transitions for Demo 0 is illustrated in the diagram below. The root path is p_0 and each view in each step is indexed, e.g., $\pi_0, \dots, \pi_4$. For brevity, the state store has been flattened as there is only a single state in Demo and closures are abbreviated. In addition, the intermediate state that triggers a retry is listed below the initial configuration $\langle \text{Demo } 0, [\text{Demo} \mapsto \dots] \rangle$.



Let's follow the execution of component Demo step by step:

- (0) The main expression is Demo 0, and the definition table δ contains a single entry of Demo.
- (1) Demo 0 evaluates into a component spec, which is initialized (STEPINIT). During initialization, the component body is evaluated in the **Init** phase (EVALMULT):
 - (a) **useState** initializes state variable s to 0 and setter function setS (STTBIND).
 - (b) Since $s = 0$, a top-level call to setS is made and **Check** decision is on (APPSETCOMP).
 - (c) **useEffect** queues the Effect body (EFF).
 - (d) Since $s \leq 1$, a unit is returned.
 Since **Check** is on, Demo's body is re-evaluated in the **Succ** phase with an empty decision and an empty effect queue (EVALONCE):
 - (a) **useState** processes the queued update, changing state s to 1 (STTREBIND). Since the state is different from the previous 0, **Effect** decision is made.
 - (b) The rest is similar to the above, so we summarize: the top-level call to setS is avoided, the Effect body is queued (EFF), and still $s = 1 \leq 1$ and hence a unit is returned.
- (2) Now we are in rendered mode $\otimes$, and we commit Effects (STEPEFFECT).
 - (a) The view decided to run **Effects**, so the queued Effect is evaluated in the **Normal** phase (COMMITEFFSPATH). Note that there is no child view's Effect to run (COMMITEFFSCONST).
 - (b) In the Effect, $s = 1$ and hence setS is called. **Check** decision is added and the state update closure is queued (APPSETNORMAL).
 - (c) **Effect** decision is removed and the decision set is now {**Check**} (COMMITEFFSPATH).
- (3) Now we are in check mode $\cup$, and we check for a re-render (STEPCHECK).
 - (a) The component body evaluates again (EVALONCE): the state update to 2 turns on **Effect** (STTBIND), the Effect is queued (EFF), and the button event handler is returned.

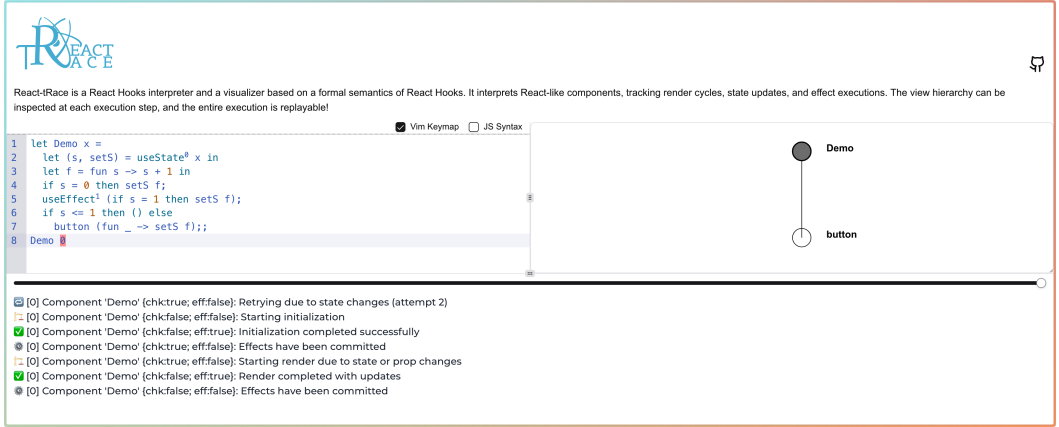


Fig. 11. The REACT-TRACE interpreter and visualizer interface, showing the illustrative example from §4.3.

- (b) The previous child $\langle \rangle$ is reconciled with the closure view spec $\langle \lambda_setS\ f, _ \rangle$ (CHECK-EFFECT).
- (c) The closure view spec is initialized as it is of different type with $\langle \rangle$ (RECONCILEOTHER).
- (4) We are again in rendered mode $\otimes$, so we commit Effects (STPEFFECT).
 - (a) Again, the view has **Effect** on and we commit the Effect (COMMITEFFSPATH), but `setS` is not called this time. Then we turn off **Effect**, leaving the decision set empty (COMMITEFFSPATH).
- (5) We are in check mode $\cup$ to check for a re-render (STEPCHECK), but this time the view is idle and we are done (CHECKIDLE).

5 The REACT-TRACE Interpreter and the Visualizer

We have implemented a definitional interpreter and a visualization tool (Fig. 11) to help developers understand the behavior of React Hooks, based on the semantics of React Hooks (§4).

The visualizer enables React programmers to examine their programs in an interactive manner:

- The view hierarchy can be inspected by visualizing the tree memory.
- The execution of the program is explained, automating the step-by-step reasoning in §4.3.
- The entire execution is replayable using a slider, allowing thorough examination of re-rendering bugs described in §3.

While our visualizer is fully functional and readily usable, we are actively developing additional features to enhance its capabilities. Planned or in-progress enhancements include more comprehensive state and trace visualization, experimental JS translation to bridge the gap between real-world React applications, and a direct preview of the rendered UI.

Implementation. The REACT-TRACE interpreter is a faithful implementation of our semantics written in OCaml 5, with the visualization front-end implemented using React with ReScript (rescript-lang.org) and TypeScript (typescriptlang.org). We used the Js_of_ocaml compiler [Vouillon and Balat 2014] to emit JS code that interfaces with our front-end.

The source code is available at [Zeta611/react-trace](https://github.com/Zeta611/react-trace), and the visualizer interface can be accessed online at react-trace.vercel.app.

6 Conformance with React

REACT-TRACE is a faithful model of Hooks backed by both theoretical and empirical evaluation:

- (1) **Conformance according to the official documentation:** Although React does not come with a formal definition, we can extract key properties of React based on the documentation. We show that REACT-TRACE respects the key properties of React as documented in §6.1.
- (2) **Direct testing against the implementation:** We present the results of empirical tests comparing REACT-TRACE against (multiple releases of) React’s runtime behavior across a range of scenarios, covering all evaluation rules, in §6.2.

6.1 Conformance Theorems on React

The React documentation provides an informal explanation of the behavioral properties of React applications. Most of these properties are explicitly captured in our semantics. To demonstrate that our semantics align with the behaviors described in the documentation, we prove two key properties in §6.1.1:

Theorem 1: When a setter function is called during rendering, React immediately attempts to re-evaluate the component body with the updated state [Meta Platforms, Inc. 2025j].

Theorem 2: A view’s Effects are executed after re-rendering triggered by a state update from itself or one of its ancestors [Meta Platforms, Inc. 2025f].

While there are slight differences between our semantics and the React runtime due to an optimization in React, this does not affect reasoning about most React applications. We have formalized this optimization and proved that it preserves output behavior (Theorem 8, §6.1.2), provided that applications adhere to the constraints specified by React.

6.1.1 Key Properties of React. The first key property states that when a setter function is called during rendering, React immediately attempts to re-evaluate the component body with the updated state. The pitfall discussed in §3.1.2 is due to this property. This behavior is modeled by allowing multiple evaluations of the component body to occur within a retrying evaluation, which happens iff a setter is applied during the first evaluation.

Theorem 1 (Re-Evaluation by Calling Setter). *The derivation of $\pi, \sigma \vdash e \Downarrow_p^\phi v', \pi'', \omega'$ includes multiple evaluations of e iff APPSETCOMP appears in the derivation of the first evaluation:*

$$\pi\{\text{dec}: \pi.\text{dec} \setminus \{\text{Check}\}, \text{effq}: []\}, \sigma \vdash e \Downarrow_p^\phi v, \pi', \omega$$

PROOF. Multiple evaluations of e occur in a retrying evaluation iff the resulting view π' from the first evaluation includes the **Check** decision (EVALMULT). Thus, it suffices to show that $\pi'.$ dec contains **Check** iff APPSETCOMP appears in the derivation of the first evaluation.

This follows directly from the semantics: APPSETCOMP is the only rule that introduces **Check** during evaluation (when $\phi \in \{\text{Init}, \text{Succ}\}$). Moreover, once added, no rule removes **Check** from the view’s decision. Therefore, π' includes **Check** iff a setter is applied during the first evaluation. ■

The second key property states that Effects are executed when queued state updates modify the state values of a view or one of its ancestors (case 1 of Theorem 2). Notably, Effects may still be executed even if the final state values remain unchanged, provided state setters are invoked during the evaluation of the component body (case 2 of Theorem 2).

Theorem 2 (Effect Execution Condition). *If $\langle t, m, \omega, \delta, \cup \rangle \hookrightarrow \langle t, m', \omega', \delta, \mu \rangle$, then for all paths p reachable from the root in both m and m' , COMMITEFFSPATH with p appears in the derivation of the next transition $\langle t, m', \omega', \delta, \mu \rangle \hookrightarrow \langle t, m'', \omega'', \delta, \mu' \rangle$ iff there exists an ancestor p' of p such that either*

- (1) *for some $\ell \in \text{dom}(m'[p']. \text{stst})$, we have $m[p']. \text{stst}[\ell]. \text{val} \neq m'[p']. \text{stst}[\ell]. \text{val}$; or*
- (2) *the derivation of $\langle t, m, \omega, \delta, \cup \rangle \hookrightarrow \langle t, m', \omega', \delta, \mu \rangle$ includes APPSETCOMP with path p' .*

PROOF. We first show only the views in rendered mode $\otimes$ may carry the **Effect** decision. This follows from the operational rules governing state transitions:

- STEPINIT transitions the initial state to rendered mode $\otimes$.
- STEPEFFECT clears all **Effect** decisions from descendants of t via *commitEfs*.
- STEP CHECK ensures **Effect** is added only when transitioning into rendered mode $\otimes$, by induction on *check*.
- STEPEVENT does not introduce **Effect**.

The COMMITEFFSPATH rule with path p appears in the derivation of the next transition iff the STEPEFFECT rule is applied (i.e., $\mu = \otimes$) and **Effect** is included in $m'[p].\text{dec}$. Among the views reachable in m' , only those updated by *check* during the first transition may carry the **Effect** decision. More precisely, **Effect** is added to p iff p is a descendant of some p' such that *check*(p') is derived via CHECKEFFECT. This holds iff the resulting view π' from the retrying evaluation of p' 's body satisfies **Effect** $\in \pi'.\text{dec}$.

We now show that **Effect** is added to such a view after the retrying evaluation iff either (a) the final state differs from the initial state, or (b) setters are applied during the evaluation.

Case (a) is immediate: state updates occur only via STTREBIND, which adds **Effect** when the new value differs from the old.

In case (b), applying a setter adds **Check** and triggers re-evaluation via EVALMULT. Resolving this re-evaluation loop requires the subsequent evaluation to yield a different result, which necessitates a different binding via STTREBIND, thereby introducing **Effect**. Hence, any terminating derivation involving APPSETCOMP includes **Effect** in the resulting decision.

Conversely, suppose **Effect** is added, but the final state values are equal to the initial ones. Then at least one state must have changed and reverted—that is, some label ℓ was updated from v to v' , and then back to v —implying multiple applications of STTREBIND with the same label. This only occurs when APPSETCOMP triggers multiple evaluations of the body. Therefore, **Effect** is added iff either some state value differs from its original value, or APPSETCOMP appears in the derivation. ■

6.1.2 Optimization in React. There is a subtle discrepancy between our semantics and the React runtime due to an optimization: When a state update callback returns the same value as the current state, React skips re-evaluating the component body.

```

1 let Counter _ =
2   print 0;
3   let (s, setS) = useState 1 in
4   [s, button (fun _ ->
5     setS (fun s -> (print 1; s));
6     setS (fun s -> (print 2; s+1));
7     setS (fun s -> (print 3; s+1)))];;
8 Counter ()
```

In our semantics, clicking the button prints $0\backslash 0\backslash 1\backslash 2\backslash 3$. The first 0 is printed during the initial render. When the button is clicked after the render, the event handler queues the state updates without printing anything. Then, during the second render triggered by the state updates, another 0 is printed. Finally, as the updates are applied, $1\backslash 2\backslash 3$ are printed.

In contrast, React prints $0\backslash 1\backslash 2\backslash 0\backslash 3$ due to the optimization. The first 0 is printed during the initial render. Upon clicking the button, the first two updates are evaluated immediately: 1 and 2 are printed as React checks whether the state has changed. The second update returns a new state, triggering a re-render. The third update is skipped at this point because a re-render is already scheduled. During the re-render, the second 0 is printed, followed by 3 from the queued third update.

We could have included this optimization for completeness, but we chose not to in order to maintain conciseness. However, this omission does not compromise correctness as long as applications conform to the constraints specified in the React documentation.

Indeed, we show that the optimization preserves program behavior in Theorem 8 (under certain conditions).

The optimization can be understood as partially applying queued state updates without re-executing the component body. Assuming these updates are pure (Definition 3) as required by the React documentation, this corresponds to a form of partial normalization, in which only a prefix of the pure updates is applied. We formalize the relationship between the original and optimized tree memory using the notion of *similarity* ($m \approx_t m'$; Definition 5), which is defined in terms of *normalization* (Definition 4)—a process of applying pure updates in advance.

Definition 3 (Purity). A closure $\langle \lambda x. e, \sigma \rangle$ is *pure* iff the application of any value v does not cause any side effects. That is, for all Σ and v , we have $\Sigma, \sigma[x \mapsto v] \vdash e \Downarrow_p^\phi _, \Sigma, []$ $\square$

Definition 4 (Normalization). A *normalization* of a state store entry is defined as the result of applying its pure prefix of state updates, with its resulting decision collected. Let l be the length of such a prefix. Then

$$\begin{aligned} \text{normalize}(\{\text{val}: v_0, \text{sttq}: [\overline{\langle \lambda x_i. e_i, \sigma_i \rangle}]_{i=1}^n\}) &= \langle \{\text{val}: v_l, \text{sttq}: [\overline{\langle \lambda x_i. e_i, \sigma_i \rangle}]_{i=l+1}^n\}, d \rangle \\ \text{where } (\pi, \sigma_i[x_i \mapsto v_{i-1}] \vdash e_i \Downarrow_p^\phi v_i, \pi, [])_{i=1}^l & \\ \text{and } d = [l \neq n \text{ ? } \{\text{Check}\} \text{ : } v_0 \neq v_l \text{ ? } \{\text{Check, Effect}\} \text{ : } \{\}] & \end{aligned}$$

We extend this notion to a state store ρ and a view π , i.e., $\text{normalize}(\rho)$ normalizing all of its entries, and $\text{normalize}(\pi)$ normalizing its state store:

$$\text{normalize}(\rho) = \left\langle \overline{[\ell \mapsto r_\ell]_{\ell \in \text{dom } \rho}}, \bigcup_{\ell \in \text{dom } \rho} d_\ell \right\rangle \quad \text{where } \text{normalize}(\rho[\ell]) = \langle r_\ell, d_\ell \rangle,$$

$$\text{normalize}(\pi) = \pi \{ \text{sttst}: \rho', \text{dec}: \pi.\text{dec} \setminus \{\text{Check}\} \cup d \} \text{ where } \langle \rho', d \rangle = \text{normalize}(\pi.\text{sttst}). \quad \square$$

Definition 5 (Similarity). Two views are *similar* iff they are equal under normalization:

$$\pi \approx \pi' \iff \text{normalize}(\pi) = \text{normalize}(\pi')$$

We extend this notion to memories, i.e., m and m' are *t-similar* iff the descendants of t in m and m' are all similar and the rest of them are equal:

$$m \approx_t m' \iff \bigwedge \begin{cases} \forall p \in \text{reachable}(m, t), m[p] \approx m'[p] \\ \forall p \notin \text{reachable}(m, t), m[p] = m'[p] \end{cases}$$

where $\text{reachable}(m, t)$ is the set of all paths in m that is reachable from the tree t . $\square$

We now state that evaluating a view similar to the original yields the same result (Lemma 7), provided the view is *valid* (Definition 6).

Definition 6 (Validity). A view is *valid* if it has only the states that are present in its component body. More precisely, π is *valid under* δ iff the domain of π 's state store is exactly the set of state labels of e , where $\pi.\text{spec} = \langle C, v \rangle$ and $\delta[C] = \langle \lambda x. e, \sigma \rangle$. We extend this notion to tree memory: A tree memory m is *valid under* δ iff its views are all valid under δ . $\square$

Note that all views encountered during execution are valid. This follows from a syntactic restriction on component bodies: Hooks must appear only at the top level. This ensures that each **useState** call in a component body is executed exactly once during initialization, so all views produced by *init* are valid.

Lemma 7 (Similar Evaluations of Component Body). *Evaluating the component body of similar views produces the same value, view, and output buffer. That is, for π valid under δ where $\pi.\text{spec} = \langle C, v \rangle$ and $\delta[C] = \langle \lambda x. e, \sigma \rangle$, if $\pi, \sigma[x \mapsto v] \vdash e \Downarrow_p^{\text{succ}} v', \pi', \omega$, we have $\hat{\pi}, \sigma[x \mapsto v] \vdash e \Downarrow_p^{\text{succ}} v', \pi', \omega$ for all $\hat{\pi} \approx \pi$.*

Table 1. Empirical validation of REACT-TRACE against React behavior.

Scenarios	Tests #	REACT-TRACE
S1 No re-render w/o a setter call	6	✓
S2 Retries ($0 < n < 25$) w/ setter call during body eval	4	✓
S3 Infinite retries ($n \geq 25$) w/ setter call during body eval	1	✓
S4 No re-render w/o Effects w/ setter call during body eval	1	✓
S5 No re-render w/ Effect w/o setter call	2	✓
S6 No re-render w/ Effect w/ id setter call	1	✓
S7 No re-render w/ Effect w/ setter calls composing to id	2	✓
S8 Re-renders ($0 < n < 100$) w/ Effect w/ setter call	16	✓
S9 Infinite re-renders ($n \geq 100$) w/ Effect w/ diverging setter call	2	✓
S10 Re-render w/ child updating parent during Effect	2	✓
S11 Re-render w/ sibling updating another during Effect	1	✓
S12 Error w/ child updating parent during body eval	1	✓
S13 Non-trivial reconciliation	4	✓
S14 No re-render w/ direct object update	1	✓
S15 Re-render w/ idle but parent updates	2	✓
S16 User event sequence	6	✓
S17 Re-render w/ setter call from user event	4	✓ [†]
S18 Recursive view hierarchy	2	✓
	38*	✓

*Some tests cover multiple scenarios.

[†]React's optimization changes some execution orders.

Building on Lemma 7, we conclude that the optimization—which replaces some views in memory with similar ones—preserves program behavior. Note that the output buffer ω may differ if component bodies perform printing, since the optimization may omit their evaluation entirely. Nevertheless, the resulting memory remains identical, regardless of whether the optimization is applied.

Theorem 8 (Similar Transitions). *If a program transitions to a state in check mode $\cup$ during execution, replacing it with a similar state results in the same final state as the original transition. That is, if $\langle e, \delta \rangle \hookrightarrow^* \langle t, m, \omega, \delta, \cup \rangle \hookrightarrow \langle t, m', \omega', \delta, \mu \rangle$, then for any $\hat{m}$ such that $m \approx_t \hat{m}$, we have $\langle t, \hat{m}, \omega, \delta, \cup \rangle \hookrightarrow \langle t, m', \omega'', \delta, \mu \rangle$. We also have $\omega' = \omega''$ when the component bodies do not print.*

Full proofs are provided in §B.

6.2 Conformance Test Suite

Our test suite, which covers all 44 evaluation rules⁵ comprising the operational semantics (§4.2), contains 38 tests that cover 18 scenarios shown in Table 1 that compare the behaviors of programs under REACT-TRACE against React's behavior. We have constructed the tests ourselves, testing the REACT-TRACE interpreter (§5) as well.

It is difficult to unit test a single evaluation rule independently because running a React program inevitably leaves footprints on multiple rules, and we have chosen the test scenarios to exhibit the pitfalls explained in §3, as well as to check the core aspects of React including render counts, side effect ordering, reconciliation, and event handling. For instance, although one of our test cases⁶ checks whether the effect queue gets flushed on retry, effectively testing rule EVALMULT (Fig. 6), it is simply categorized as S2 and S8 in Table 1.

⁵See §A.2 for the complete set of rules.

⁶effect_queue_gets_flushed_on_retry

Notably, we test the boundaries of React that even experienced developers may find confusing. For instance, S12 in Table 1 covers attempting to update a parent’s state during the evaluation of its child, which correctly produces an error in both implementations. Other scenarios include unnecessary re-rendering that modifies the view hierarchy (S8–S11, Table 1; similar to examples in §3.2), constructing complex UI with recursive components (S18, Table 1), and more.

The tests consist of equivalent components implemented twice—once in REACT-TRACE and once in React—then comparing that they exhibit the same behavior. For example, to empirically check infinite retries (S3, Table 1) on the React-side, we install an error boundary [Meta Platforms, Inc. 2025a] to catch an exception when a component reaches React’s hard-coded limit of 25 retries. On the REACT-TRACE-side, we set the retry threshold to 25 and see whether the component did not stop rendering. For tests measuring the number of render cycles on the React-side, we indirectly measure the count by counting the prints inside an Effect.

Almost all tests show identical behavior between REACT-TRACE and React’s runtime, with one minor difference due to the optimization performed by React as discussed in §6.1.2. We might as well include this optimization in REACT-TRACE by adding a special flag and modifying APPSETNORMAL, but we deliberately chose not to in order to keep the clarity of our semantics.

REACT-TRACE does not model a specific version of React: We have tested against the latest versions of every major React release since Hooks were introduced in 16.8 (Feb 2019). All test cases have been reproduced in versions 16.14.0 (Oct 2020), 17.0.2 (Mar 2021), 18.3.1 (Apr 2024), and 19.1.0 (Mar 2025).

The test suite is available alongside the REACT-TRACE interpreter at [Zeta611/react-trace](https://github.com/Zeta611/react-trace).

7 Discussion

Extensions to Other Hooks. While our semantics focuses on the two most prevalent Hooks **useState** and **useEffect**, it’s designed to be extensible. Incorporating additional Hooks builds upon existing machinery for bookkeeping and lifecycle modes.

Hooks are mostly state managing (**useState**, **useRef**, **useMemo**, **useContext**) or side effect performing (**useEffect**, **useLayoutEffect**, **useInsertionEffect**), which can be supported by extending view record π and/or inserting modes in render transitions. Users can also build custom Hooks by combining existing Hooks in a function, which should be **use**-prefixed.

We sketch how to extend the semantics to other Hooks:

useRef^f Add $\pi.\text{refst}$ storing refs $[\ell \mapsto v]$. Unlike **useState**, mutating a ref does not trigger re-renders, so no **Check** decision is added when mutated. In **Init** phase, the ref is initialized; in **Succ** phase, the same ref is returned.

useMemo^f Add $\pi.\text{memost}$ storing values and their dependencies $[\ell \mapsto \{\text{val} : v, \text{deps} : [\bar{v}]\}]$. Similar to **useState**, in **Init** the value is computed from the provided function; in **Succ** the value is recomputed only when dependencies differ from the stored ones.

useContext Add $\pi.\text{ctxst}$ storing a list of consumed contexts and tree memory m is traversed upward to find the nearest context provider. When a provider’s value changes, all consuming views are marked with **Check** to trigger re-renders.

useLayoutEffect Add mode $\otimes'$ into render transitions (Fig. 4). STEPINIT enters $\otimes'$ after the initial render; a new rule STEPLAYOUTEFFECT processes LayoutEffects determining the transition $\langle t, m, \omega, \delta, \otimes' \rangle \hookrightarrow \langle t, m', \omega', \delta, \cup \rangle$.

useInsertionEffect Similar to **useLayoutEffect** but runs before. State updates are forbidden in InsertionEffects by the React runtime, hence the pitfalls described in §3 cannot occur.

Custom Hooks Supported by allowing user function definitions (Fig. 1) and DefTable (§4.1). No additional machinery is required as custom Hooks are compositions of primitive Hooks.

Verifying the React Compiler. Our formalization of Hooks is particularly timely as the React team is currently developing the React compiler [Meta Platforms, Inc. 2025d], where various optimizations can be verified using our semantics as in §6.1.2. The compiler optimizes React programs by eliminating unnecessary re-renders through memoization. However, without a proper formal semantics of Hooks, there is no rigorous way to verify that the compiler preserves program behavior. Our semantics can thus serve as a foundation for reasoning about their correctness.

Beyond React. Our work can be extended to accommodate other reactive UI frameworks as well. There is a plethora of GUI frameworks—React, Preact (preactjs.com), Dioxus (dioxuslabs.com), Solid (solidjs.com), Svelte (svelte.dev), and Leptos (leptos.dev)—with variations in their reactivity models.

These frameworks can be categorized based on three properties: (a) whether they re-read component specifications for re-rendering, (b) how state updates are processed (queued or immediate), and (c) which reactivity primitives they employ.

Reactive primitives include Hooks (where frameworks schedule update checks), signals (where state updates propagate actively), and compiler-assisted methods (where dependencies are statically resolved). By parameterizing our semantics with appropriate timing and semantic operators, we capture these variations. A full comparison of the frameworks is provided in §C.

As a reference to how similar these frameworks can be, compare the Dioxus code on the left (where Rules of Hooks apply as well! [Dioxus Labs 2024]) and the REACT-TRACE code on the right.

<pre>1 pub fn Counter() -> Element { 2 let mut count = use_signal(0); 3 rsx! { button { onclick: move _ 4 count += 1, "{count}" } }</pre>	<pre>1 let Counter _ = 2 let (count, setCount) = useState 0 in 3 [button (fun _ -> 4 setCount(fun count -> count+1)), count]</pre>
---	--

8 Related Work

Research on React. Our work presents the first formal semantics for React’s function components and Hooks, which have become the standard for modern React applications. Moreover, we deliberately decouple the semantics of React and Hooks from JS, enabling reasoning about Hooks independently from the host language. This approach contrasts with previous work on classed-based React components by Madsen et al. [2020], who formalized a core calculus λ_{react} upon λ_{js} [Guha et al. 2010] that captures React’s component lifecycle and reconciliation.

While our research aims to faithfully model the actual React runtime, Crichton and Krishnamurthi [2024] took a more general approach with their document calculus. Their semantics for general document languages demonstrates how features like document references interact with documents written in a React-like language using a simplified runtime, rather than attempting to capture the full complexity of React’s behavior.

React Development Tools. REACT-TRACE provides a theoretical foundation for building practical tools grounded on the semantics of Hooks, unlike existing approaches limited to runtime detection or syntactic checking. This semantics-based approach contrasts with tools like ESLint (eslint.org), a *de facto* standard static analyzer that syntactically checks code to prevent basic errors when writing React components. Similarly, runtime libraries such as why-did-you-render ([welldone-software/why-did-you-render](https://github.com/welldone-software/why-did-you-render)) and React Scan (react-scan.com) detect (re-)renders by monkey patching and inspecting the React library at runtime but lack the formal framework to explain why these behaviors occur.

Functional Reactive GUI Frameworks. React borrows concepts from functional reactive programming (FRP) [Elliott and Hudak 1997] but implements them quite differently. While React embraces declarative components, it requires explicit state management via **useState** and provides escape hatches through **useEffect**. This differs significantly from FRP web frameworks which use signals

as reactive primitives. Notable FRP web frameworks include Flapjax [Meyerovich et al. 2009], Ur/Web [Chlipala 2015a,b], and the early versions of Elm [Czaplicki and Chong 2013].

The Elm Architecture. The Elm programming language (elm-lang.org) has moved on from FRP and settled on *The Elm Architecture (TEA)* or the *Model-View-Update (MVU)* pattern, which has been formalized as λ_{MVU} [Fowler 2020]. In TEA, there is a clear separation between a model and a view, where a message is dispatched from the view to update the model through an update function, creating a unidirectional data flow [Feldman 2020]. One can follow a similar pattern in React if a state management library like Redux (redux.js.org) is used.

Multi-tier Programming. In practice, React applications span multiple *tiers*—client, server, and database—requiring developers to manually maintain coherency between them. Full stack React frameworks such as Next.js (nextjs.org) and Remix (remix.run) borrow ideas from *tierless* or *multi-tier* (MT) languages to allow developers to develop both the client- and server-side in a single program. MT languages like Ur/Web [Chlipala 2015a,b] (an ML-like language with whole-program optimization), ELIOM [Radanne et al. 2016a,b] (an extension of OCaml focusing on modularity), Hop [Boudol et al. 2012; Serrano et al. 2006] (a dynamically typed Scheme-based framework), and Links [Cooper et al. 2007] (an ML-like language with an effect system that compiles to SQL and JS) enable higher-level reasoning about the whole system while providing stronger safety guarantees [Weisenburger et al. 2020].

JavaScript and Web Semantics. Our work extends the rich tradition of formalizing web technologies by providing a formal model of React Hooks, complementing existing work on web semantics. Numerous formal models of JS have been proposed [Fragoso Santos et al. 2017; Guha et al. 2010; Madsen et al. 2017; Maffei et al. 2008; Park et al. 2015; Politz et al. 2012], including the recent work on mechanized extraction of the prose ECMA-262 specification by Park et al. [2021]. WebSpec [Veronese et al. 2023] provides a formal browser model in Rocq (formerly known as Coq) to verify browser security. Several works [Pancheekha et al. 2019, 2018; Pancheekha and Torlak 2016] have formalized the browser layout algorithm and the CSS semantics using SMT-based encoding to verify web page layouts.

9 Conclusion and Future Work

We have presented REACT-TRACE, an operational semantics for helping React developers understand the behavior of Hooks. Render-related UI bugs often puzzle developers due to the peculiar semantics of Hooks. We have captured the essence of Hooks in REACT-TRACE, which clearly explains subtle pitfalls that occur when multiple Hooks interact with each other. Moreover, developers can use our REACT-TRACE-interpreter and visualizer to inspect how their programs render step-by-step, making the opaque render process transparent.

As a foundation for future work, REACT-TRACE opens several research directions. First, our semantics enables semantic-based static analyzers and type systems that go beyond the syntactic or runtime checks of current tools, catching subtle bugs at compile time. Second, as React evolves with features like React compiler and server components, REACT-TRACE provides a formal basis for verifying their correctness. Third, REACT-TRACE can be refined for pedagogical use in building a correct mental model of Hooks. Finally, integrating REACT-TRACE with existing web semantics can enable end-to-end verification of web programs, from component behavior to final rendering.

Data-Availability Statement

The source code and test suite for the REACT-TRACE interpreter and visualizer are available on GitHub at [Zeta611/react-trace](https://github.com/Zeta611/react-trace), archived on Zenodo [Lee and Ahn 2025], and hosted online at react-trace.vercel.app.

Acknowledgments

We are grateful to Joonhyup Lee, Gyuhyeok Oh, Lauren Minjung Kwon, and Hyeongseo Yoo for valuable comments, Will Crichton for early feedback, and Sukyoung Ryu for helpful suggestions. We thank Susana Angélica Balderas Chiw for the logo in Fig. 11. This work was supported by BK21 FOUR Intelligence Computing (Dept. of Computer Science and Engineering, SNU) funded by National Research Foundation of Korea (NRF) (4199990214639), Greenlabs Co., Ltd. (0536-20220078), and Samsung Electronics Co., Ltd. (0536-20230088).

References

- Gérard Boudol, Zhengqin Luo, Tamara Rezk, and Manuel Serrano. 2012. Reasoning about Web Applications: An Operational Semantics for HOP. *ACM Trans. Program. Lang. Syst.* 34, 2, Article 10 (June 2012), 40 pages. doi:10.1145/2220365.2220369
- Adam Chlipala. 2015a. An Optimizing Compiler for a Purely Functional Web-Application Language. In *Proceedings of the 20th ACM SIGPLAN International Conference on Functional Programming* (Vancouver, BC, Canada) (ICFP '15). Association for Computing Machinery, New York, NY, USA, 10–21. doi:10.1145/2784731.2784741
- Adam Chlipala. 2015b. Ur/Web: A Simple Model for Programming the Web. In *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Mumbai, India) (POPL '15). Association for Computing Machinery, New York, NY, USA, 153–165. doi:10.1145/2676726.2677004
- Ezra Cooper, Sam Lindley, Philip Wadler, and Jeremy Yallop. 2007. Links: Web Programming Without Tiers. In *Formal Methods for Components and Objects (FMCO '06) (Lecture Notes in Computer Science, Vol. 4709)*, Frank S. de Boer, Marcello M. Bonsangue, Susanne Graf, and Willem-Paul de Roever (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 266–296. doi:10.1007/978-3-540-74792-5_12
- Patrick Cousot and Radhia Cousot. 1977. Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages* (Los Angeles, California) (POPL '77). Association for Computing Machinery, New York, NY, USA, 238–252. doi:10.1145/512950.512973
- Will Crichton and Shriram Krishnamurthi. 2024. A Core Calculus for Documents: Or, Lambda: The Ultimate Document. *Proc. ACM Program. Lang.* 8, POPL, Article 23 (Jan. 2024), 28 pages. doi:10.1145/3632865
- Evan Czaplicki and Stephen Chong. 2013. Asynchronous Functional Reactive Programming for GUIs. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Seattle, Washington, USA) (PLDI '13). Association for Computing Machinery, New York, NY, USA, 411–422. doi:10.1145/2491956.2462161
- Dioxus Labs. 2024. Hooks and component state. <https://dioxuslabs.com/learn/0.6/reference/hooks/>. Accessed: 2025-03-25.
- Conal Elliott and Paul Hudak. 1997. Functional Reactive Animation. In *Proceedings of the Second ACM SIGPLAN International Conference on Functional Programming* (Amsterdam, The Netherlands) (ICFP '97). Association for Computing Machinery, New York, NY, USA, 263–273. doi:10.1145/258948.258973
- Richard Feldman. 2020. *Elm in Action*. Manning Publications, Shelter Island, NY. 344 pages. <https://www.manning.com/books/elm-in-action>
- Simon Fowler. 2020. Model-View-Update-Communicate: Session Types Meet the Elm Architecture. In *34th European Conference on Object-Oriented Programming (ECOOP 2020) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 166)*, Robert Hirschfeld and Tobias Pape (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 14:1–14:28. doi:10.4230/LIPIcs.ECOOP.2020.14
- José Fragoso Santos, Petar Maksimović, Daiva Naudžiūnienė, Thomas Wood, and Philippa Gardner. 2017. JaVerT: JavaScript Verification Toolchain. *Proc. ACM Program. Lang.* 2, POPL, Article 50 (Dec. 2017), 33 pages. doi:10.1145/3158138
- Arjun Guha, Claudiu Saftoiu, and Shriram Krishnamurthi. 2010. The Essence of JavaScript. In *ECOOP 2010 – Object-Oriented Programming (Lecture Notes in Computer Science, Vol. 6183)*, Theo D'Hondt (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 126–150. doi:10.1007/978-3-642-14107-2_7
- Gilles Kahn. 1987. Natural Semantics. In *STACS 87 (Lecture Notes in Computer Science, Vol. 247)*, Franz J. Brandenburg, Guy Vidal-Naquet, and Martin Wirsing (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 22–39. doi:10.1007/BFb0039592
- Jay Lee. 2025. A screenshot of the StackOverflow search results of "useEffect" "infinite". <https://archive.org/details/useeffect-infinite>. Accessed: 2025-03-24.

- Jay Lee and Joongwon Ahn. 2025. *Artifact for “REACT-TRACE: A Semantics for Understanding React Hooks”*. doi:10.5281/zenodo.16916356
- Magnus Madsen, Ondřej Lhoták, and Frank Tip. 2017. A Model for Reasoning About JavaScript Promises. *Proc. ACM Program. Lang.* 1, OOPSLA, Article 86 (Oct. 2017), 24 pages. doi:10.1145/3133910
- Magnus Madsen, Ondřej Lhoták, and Frank Tip. 2020. A Semantics for the Essence of React. In *34th European Conference on Object-Oriented Programming (ECOOP 2020) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 166)*, Robert Hirschfeld and Tobias Pape (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 12:1–12:26. doi:10.4230/LIPIcs.ECOOP.2020.12
- Sergio Maffei, John C. Mitchell, and Ankur Taly. 2008. An Operational Semantics for JavaScript. In *Programming Languages and Systems (APLAS ’08) (Lecture Notes in Computer Science, Vol. 5356)*, Ganesan Ramalingam (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 307–325. doi:10.1007/978-3-540-89330-1_22
- Meta Platforms, Inc. 2022. JSX. <https://facebook.github.io/jsx/>. Accessed: 2025-03-24.
- Meta Platforms, Inc. 2025a. Component – React. <https://react.dev/reference/react/Component>. Accessed: 2025-03-12.
- Meta Platforms, Inc. 2025b. Higher-Order Components – React. <https://legacy.reactjs.org/docs/higher-order-components.html>. Accessed: 2025-03-18.
- Meta Platforms, Inc. 2025c. Queueing a Series of State Updates – React. <https://react.dev/learn/queueing-a-series-of-state-updates>. Accessed: 2025-03-12.
- Meta Platforms, Inc. 2025d. React Compiler – React. <https://react.dev/learn/react-compiler>. Accessed: 2025-03-25.
- Meta Platforms, Inc. 2025e. Reconciliation – React. <https://legacy.reactjs.org/docs/reconciliation.html>. Accessed: 2025-07-23.
- Meta Platforms, Inc. 2025f. Render and Commit – React. <https://react.dev/learn/render-and-commit>. Accessed: 2025-03-25.
- Meta Platforms, Inc. 2025g. Rules of Hooks – React. <https://react.dev/reference/rules/rules-of-hooks>. Accessed: 2025-03-12.
- Meta Platforms, Inc. 2025h. Rules of React – React. <https://react.dev/reference/rules>. Accessed: 2025-03-12.
- Meta Platforms, Inc. 2025i. Synchronizing with Effects – React. <https://react.dev/learn/synchronizing-with-effects>. Accessed: 2025-03-12.
- Meta Platforms, Inc. 2025j. useState – React. <https://react.dev/reference/react/useState>. Accessed: 2025-03-25.
- Leo A. Meyerovich, Arjun Guha, Jacob Baskin, Gregory H. Cooper, Michael Greenberg, Aleks Bromfield, and Shriram Krishnamurthi. 2009. Flapjax: A Programming Language for Ajax Applications. In *Proceedings of the 24th ACM SIGPLAN Conference on Object Oriented Programming Systems Languages and Applications* (Orlando, Florida, USA) (OOPSLA ’09). Association for Computing Machinery, New York, NY, USA, 1–20. doi:10.1145/1640089.1640091
- Pavel Panchekha, Michael D. Ernst, Zachary Tatlock, and Shoaib Kamil. 2019. Modular Verification of Web Page Layout. *Proc. ACM Program. Lang.* 3, OOPSLA, Article 151 (Oct. 2019), 26 pages. doi:10.1145/3360577
- Pavel Panchekha, Adam T. Geller, Michael D. Ernst, Zachary Tatlock, and Shoaib Kamil. 2018. Verifying That Web Pages Have Accessible Layout. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Philadelphia, PA, USA) (PLDI ’18). Association for Computing Machinery, New York, NY, USA, 1–14. doi:10.1145/3192366.3192407
- Pavel Panchekha and Emina Torlak. 2016. Automated Reasoning for Web Page Layout. In *Proceedings of the 2016 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications* (Amsterdam, Netherlands) (OOPSLA ’16). Association for Computing Machinery, New York, NY, USA, 181–194. doi:10.1145/2983990.2984010
- Daejun Park, Andrei Stănescu, and Grigore Roşu. 2015. KJS: A Complete Formal Semantics of JavaScript. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Portland, OR, USA) (PLDI ’15). Association for Computing Machinery, New York, NY, USA, 346–356. doi:10.1145/2737924.2737991
- Jihyeok Park, Jihee Park, Seungmin An, and Sukyoung Ryu. 2021. JISET: JavaScript IR-based Semantics Extraction Toolchain. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering* (Virtual Event, Australia) (ASE ’20). Association for Computing Machinery, New York, NY, USA, 647–658. doi:10.1145/3324884.3416632
- Joe Gibbs Politz, Matthew J. Carroll, Benjamin S. Lerner, Justin Pombrio, and Shriram Krishnamurthi. 2012. A Tested Semantics for Getters, Setters, and Eval in JavaScript. In *Proceedings of the 8th Symposium on Dynamic Languages* (Tucson, Arizona, USA) (DLS ’12). Association for Computing Machinery, New York, NY, USA, 1–16. doi:10.1145/2384577.2384579
- Pranjal. 2019. The useState set method is not reflecting a change immediately. <https://stackoverflow.com/q/54069253>. Accessed: 2025-03-24.
- Gabriel Radanne, Vasilis Papavasileiou, Jérôme Vouillon, and Vincent Balat. 2016a. Eliom: tierless Web programming from the ground up. In *Proceedings of the 28th Symposium on the Implementation and Application of Functional Programming Languages* (Leuven, Belgium) (IFL ’16). Association for Computing Machinery, New York, NY, USA, Article 8, 12 pages. doi:10.1145/3064899.3064901
- Gabriel Radanne, Jérôme Vouillon, and Vincent Balat. 2016b. Eliom: A Core ML Language for Tierless Web Programming. In *Programming Languages and Systems*, Atsushi Igarashi (Ed.). Springer International Publishing, Cham, 377–397.

- Xavier Rival and Kwangkeun Yi. 2020. *Introduction to Static Analysis: An Abstract Interpretation Perspective*. The MIT Press, Cambridge, Massachusetts.
- Sukyoung Ryu and Jihyeok Park. 2024. JavaScript Language Design and Implementation in Tandem. *Commun. ACM* 67, 5 (May 2024), 86–95. doi:10.1145/3624723
- Manuel Serrano, Erick Galesio, and Florian Loitsch. 2006. Hop, a Language for Programming the Web 2.0. In *Companion to the 21st ACM SIGPLAN Symposium on Object-Oriented Programming Systems, Languages, and Applications* (Portland, Oregon, USA) (OOPSLA '06). Association for Computing Machinery, New York, NY, USA, 975–985.
- Stack Exchange, Inc. 2024. 2024 Stack Overflow Developer Survey. <https://survey.stackoverflow.co/2024/>. Accessed: 2025-01-21.
- CACM Staff. 2016. React: Facebook’s Functional Turn on Writing JavaScript. *Commun. ACM* 59, 12 (Dec. 2016), 56–62. doi:10.1145/2980991
- Quoc Van Tang. 2019. React hooks useState setValue still rerender one more time when value is equal. <https://stackoverflow.com/q/57652176>. Accessed: 2025-03-24.
- Tehila. 2022. Updating state to the same value directly in the component body during render causes infinite loop. <https://stackoverflow.com/q/74034014>. Accessed: 2025-03-24.
- vadirn. 2018. Does React batch state update functions when using hooks? <https://stackoverflow.com/q/53048495>. Accessed: 2025-03-24.
- Lorenzo Veronese, Benjamin Farinier, Pedro Bernardo, Mauro Tempesta, Marco Squarcina, and Matteo Maffei. 2023. WebSpec: Towards Machine-Checked Analysis of Browser Security Mechanisms. In *2023 IEEE Symposium on Security and Privacy (SP '23)*. IEEE, San Francisco, CA, USA, 2761–2779. doi:10.1109/sp46215.2023.10179465
- Jérôme Vouillon and Vincent Balat. 2014. From bytecode to JavaScript: the Js_of_ocaml compiler. *Softw. Pract. Exper.* 44, 8 (Aug. 2014), 951–972. doi:10.1002/spe.2187
- Pascal Weisenburger, Johannes Wirth, and Guido Salvaneschi. 2020. A Survey of Multitier Programming. *ACM Comput. Surv.* 53, 4, Article 81 (Sept. 2020), 35 pages. doi:10.1145/3397495

A Full Operational Semantics of REACT-TRACE

A.1 Semantic Domains

$\text{Val } v ::= k$	constant values
$\quad cl$	closure
$\quad C$	component name
$\quad cs$	component spec
$\quad [\bar{s}]$	view spec list
$\quad \langle \ell, p \rangle$	setter closure
$\text{Const } k ::= \langle \rangle \mid \mathbf{tt} \mid \mathbf{ff} \mid n$	constant values
$\text{Clos } cl ::= \langle \lambda x.e, \sigma \rangle$	closure
$\text{Env } \sigma ::= [x \mapsto v]$	environment
$\text{ComSpec } cs ::= \langle C, v \rangle$	component spec
$\text{ViewSpec } s ::= k \mid cl \mid [\bar{s}] \mid cs$	view spec
$\text{DefTable } \delta ::= [\overline{C \mapsto \lambda x.e}]$	component def table
$\quad \ell \in \mathbb{N}$	label
$\quad p \in \text{Path}$	tree path
$\text{TreeMem } m ::= [p \mapsto \pi]$	tree memory
$\text{View } \pi ::= \{\text{spec: } cs, \text{dec: } \{\bar{d}\}, \text{sttst: } \rho, \text{effq: } q, \text{child: } t\}$	state environment
$\text{Decision } d ::= \text{Check} \mid \text{Effect}$	decision
$\text{Tree } t ::= k \mid cl \mid [\bar{t}] \mid p$	tree
$\text{SttStore } \rho ::= [\ell \mapsto \{\text{val: } v, \text{sttq: } q\}]$	state store
$\text{JobQ } q ::= [\bar{cl}]$	job queue
$\text{Context } \Sigma ::= m \mid \pi$	evaluation context
$\text{Phase } \phi ::= \text{Init} \mid \text{Succ} \mid \text{Normal}$	phase
$\text{Mode } \mu ::= \otimes (\text{rendered}) \mid \cup (\text{check}) \mid \bullet (\text{event loop})$	runtime mode

A.2 Operational Semantics

$$\langle e, \delta \rangle \text{ or } \langle t, m, \omega, \delta, \mu \rangle \hookrightarrow \langle t, m', \omega', \delta, \mu' \rangle$$

STEPINIT

$$\frac{[], [] \vdash e \Downarrow_{\text{Normal}} s, [], \omega \quad [] \vdash \text{init}(s) = \langle t, m, \omega' \rangle}{\langle e, \delta \rangle \hookrightarrow \langle t, m, \omega \dashv \omega', \delta, \otimes \rangle}$$

STPEFFECT

$$\frac{m \vdash \text{commitEffs}(t) = \langle m', \omega' \rangle}{\langle t, m, \omega, \delta, \otimes \rangle \hookrightarrow \langle t, m', \omega \dashv \omega', \delta, \cup \rangle}$$

STEPCHECK

$$\frac{m, \delta \vdash \text{check}(t) = \langle \mu, m', \omega' \rangle}{\langle t, m, \omega, \delta, \cup \rangle \hookrightarrow \langle t, m', \omega \dashv \omega', \delta, \mu \rangle}$$

STPEVENT

$$\frac{\langle \lambda x.e, \sigma \rangle \in \text{handlers}(m, t) \quad m, \sigma[x \mapsto \langle \rangle] \vdash e \Downarrow_{\text{Normal}} v, m', \omega'}{\langle t, m, \omega, \delta, \bullet \rangle \hookrightarrow \langle t, m', \omega \dashv \omega', \delta, \cup \rangle}$$

$$\boxed{\text{handlers}(m, t) = \{\overline{cl}\}}$$

$$\text{handlers}(m, t) = \begin{cases} \{\} & \text{if } t = k \\ \{cl\} & \text{if } t = cl \\ \bigcup_{i=1}^n \text{handlers}(m, t_i) & \text{if } t = [\overline{t_i}]_{i=1}^n \\ \text{handlers}(m, m[p].\text{children}) & \text{if } t = p \end{cases}$$

$$\boxed{\Sigma, \sigma \vdash e \Downarrow_p^\phi v, \Sigma', \omega}$$

UNIT

$$\frac{}{\Sigma, \sigma \vdash () \Downarrow_p^\phi \langle \rangle, \Sigma, []}$$

TRUE

$$\frac{}{\Sigma, \sigma \vdash \text{true} \Downarrow_p^\phi \mathbf{tt}, \Sigma, []}$$

FALSE

$$\frac{}{\Sigma, \sigma \vdash \text{false} \Downarrow_p^\phi \mathbf{ff}, \Sigma, []}$$

INT

$$\frac{}{\Sigma, \sigma \vdash n \Downarrow_p^\phi n, \Sigma, []}$$

VAR

$$\frac{}{\Sigma, \sigma \vdash x \Downarrow_p^\phi \sigma(x), \Sigma, []}$$

BOP

$$\frac{\Sigma, \sigma \vdash e_1 \Downarrow_p^\phi n_1, \Sigma_1, \omega_1 \quad \Sigma_1, \sigma \vdash e_2 \Downarrow_p^\phi n_2, \Sigma_2, \omega_2 \quad f_\oplus \in \mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{Z}}{\Sigma, \sigma \vdash e_1 \oplus e_2 \Downarrow_p^\phi f_\oplus(n_1, n_2), \Sigma_2, \omega_1 \uplus \omega_2}$$

COND

$$\frac{\Sigma, \sigma \vdash e_1 \Downarrow_p^\phi b, \Sigma', \omega \quad \Sigma', \sigma \vdash (b \text{ ? } e_2 \text{ : } e_3) \Downarrow_p^\phi v, \Sigma'', \omega'}{\Sigma, \sigma \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Downarrow_p^\phi v, \Sigma'', \omega \uplus \omega'}$$

FUNC

$$\frac{}{\Sigma, \sigma \vdash \text{fun } x \rightarrow e \Downarrow_p^\phi \langle \lambda x. e, \sigma \rangle, \Sigma, []}$$

SEQ

$$\frac{\Sigma, \sigma \vdash e_1 \Downarrow_p^\phi _, \Sigma_1, \omega_1 \quad \Sigma_1, \sigma \vdash e_2 \Downarrow_p^\phi v_2, \Sigma_2, \omega_2}{\Sigma, \sigma \vdash e_1 ; e_2 \Downarrow_p^\phi v_2, \Sigma_2, \omega_1 \uplus \omega_2}$$

LIST

$$\frac{(\Sigma_{i-1}, \sigma \vdash e_i \Downarrow_p^\phi s_i, \Sigma_i, \omega_i)_{i=1}^n}{\Sigma_0, \sigma \vdash [\overline{e_i}]_{i=1}^n \Downarrow_p^\phi [\overline{s_i}]_{i=1}^n, \Sigma_n, \uplus_{i=1}^n \omega_i}$$

LETBIND

$$\frac{\Sigma, \sigma \vdash e_1 \Downarrow_p^\phi v_1, \Sigma_1, \omega_1 \quad \Sigma_1, \sigma[x \mapsto v_1] \vdash e_2 \Downarrow_p^\phi v_2, \Sigma_2, \omega_2}{\Sigma, \sigma \vdash \text{let } x = e_1 \text{ in } e_2 \Downarrow_p^\phi v_2, \Sigma_2, \omega_1 \uplus \omega_2}$$

APPFUNC

$$\frac{\Sigma, \sigma \vdash e_1 \Downarrow_p^\phi \langle \lambda x. e, \sigma' \rangle, \Sigma_1, \omega_1 \quad \Sigma_1, \sigma \vdash e_2 \Downarrow_p^\phi v_2, \Sigma_2, \omega_2 \quad \Sigma_2, \sigma'[x \mapsto v_2] \vdash e \Downarrow_p^\phi v', \Sigma', \omega'}{\Sigma, \sigma \vdash e_1 e_2 \Downarrow_p^\phi v', \Sigma', \omega_1 \uplus \omega_2 \uplus \omega'}$$

APPCOM

$$\frac{\Sigma, \sigma \vdash e_1 \Downarrow_p^\phi C, \Sigma_1, \omega_1 \quad \Sigma_1, \sigma \vdash e_2 \Downarrow_p^\phi v, \Sigma_2, \omega_2}{\Sigma, \sigma \vdash e_1 e_2 \Downarrow_p^\phi \langle C, v \rangle, \Sigma_2, \omega_1 \uplus \omega_2}$$

APPSETCOMP

$$\frac{\pi, \sigma \vdash e_1 \Downarrow_p^\phi \langle \ell, p \rangle, \pi_1, \omega_1 \quad \pi_1, \sigma \vdash e_2 \Downarrow_p^\phi cl, \pi_2, \omega_2 \quad \phi \in \{\text{Init}, \text{Succ}\}}{\pi, \sigma \vdash e_1 e_2 \Downarrow_p^\phi \langle \rangle, \pi_2 \left\{ \begin{array}{l} \text{let } \rho = \pi_2.\text{sttst}, vq = \rho[\ell] \text{ in} \\ \text{dec: } \pi_2.\text{dec} \cup \{\text{Check}\} \\ \text{sttst: } \rho[\ell \mapsto vq\{\text{sttq: } vq.\text{sttq} \uplus [cl]\}] \end{array} \right\}, \omega_1 \uplus \omega_2}$$

APPSETNORMAL

$$\frac{m, \sigma \vdash e_1 \Downarrow_{-}^{\text{Normal}} \langle \ell, p \rangle, m_1, \omega_1 \quad m_1, \sigma \vdash e_2 \Downarrow_{-}^{\text{Normal}} cl, m_2, \omega_2}{m, \sigma \vdash e_1 e_2 \Downarrow_{-}^{\text{Normal}} \langle \rangle, m_2 \left[\begin{array}{l} \text{let } \pi = m_2[p], \rho = \pi.\text{sttst}, vq = \rho[\ell] \text{ in} \\ p \mapsto \pi \left\{ \begin{array}{l} \text{dec: } \pi.\text{dec} \cup \{\text{Check}\} \\ \text{sttst: } \rho[\ell \mapsto vq\{\text{sttq: } vq.\text{sttq} \vdash [cl]\}] \end{array} \right\} \end{array} \right], \omega_1 \vdash \omega_2}$$

STTBIND

$$\frac{\pi, \sigma \vdash e_1 \Downarrow_p^{\text{Init}} v_1, \pi_1, \omega_1 \quad \pi_1 \left\{ \begin{array}{l} \text{sttst: } \pi_1.\text{sttst} \left[\ell \mapsto \left\{ \begin{array}{l} \text{val: } v_1 \\ \text{sttq: } [] \end{array} \right\} \right] \\ \sigma \left[\begin{array}{l} x \mapsto v_1 \\ x_{\text{set}} \mapsto \langle \ell, p \rangle \end{array} \right] \end{array} \right\} \vdash e_2 \Downarrow_p^{\text{Init}} v_2, \pi_2, \omega_2}{\pi, \sigma \vdash \text{let } (x, x_{\text{set}}) = \text{useState}^\ell e_1 \text{ in } e_2 \Downarrow_p^{\text{Init}} v_2, \pi_2, \omega_1 \vdash \omega_2}$$

STTREBIND

$$\frac{\pi_0.\text{sttst}[\ell] = \{ \text{val: } v_0, \text{sttq: } [\langle \lambda x_i. e'_i, \sigma_i \rangle]_{i=1}^n \} \quad (\pi_{i-1}, \sigma_i[x_i \mapsto v_{i-1}] \vdash e'_i \Downarrow_p^\phi v_i, \pi_i, \omega_i)_{i=1}^n \quad \pi_n \left\{ \begin{array}{l} \text{dec: } \pi_n.\text{dec} \cup \{ [v_n \neq v_0 \text{ ? } \text{Effect} \text{ ? } \{ \}] \} \\ \text{sttst: } \pi_n.\text{sttst}[\ell \mapsto \{ \text{val: } v_n, \text{sttq: } [] \}] \end{array} \right\}, \sigma \left[\begin{array}{l} x \mapsto v_n \\ x_{\text{set}} \mapsto \langle \ell, p \rangle \end{array} \right] \vdash e_2 \Downarrow_p^\phi v, \pi', \omega'}{\pi_0, \sigma \vdash \text{let } (x, x_{\text{set}}) = \text{useState}^\ell e_1 \text{ in } e_2 \Downarrow_p^{\text{Succ}} v, \pi', (\vdash_{i=0}^n \omega_i) \vdash \omega'}$$

EFF

$$\frac{\phi \in \{\text{Init}, \text{Succ}\}}{\pi, \sigma \vdash \text{useEffect } e \Downarrow_p^\phi \langle \rangle, \pi\{\text{effq: } \pi.\text{effq} \vdash [\langle \lambda_{-}. e, \sigma \rangle]\}, []}$$

PRINT

$$\frac{\Sigma, \sigma \vdash e \Downarrow_p^\phi v, \Sigma', \omega}{\Sigma, \sigma \vdash \text{print } e \Downarrow_p^\phi \langle \rangle, \Sigma', \omega \vdash [\omega]}$$

$$\boxed{\pi, \sigma \vdash e \Downarrow_p^\phi s, \pi', \omega}$$

EVALONCE

$$\frac{\pi\{\text{dec: } \pi.\text{dec} \setminus \{\text{Check}\}, \text{effq: } []\}, \sigma \vdash e \Downarrow_p^\phi s, \pi', \omega \quad \text{Check} \notin \pi'.\text{dec} \quad \phi \in \{\text{Init}, \text{Succ}\}}{\pi, \sigma \vdash e \Downarrow_p^\phi s, \pi', \omega}$$

EVALMULT

$$\frac{\pi\{\text{dec: } \pi.\text{dec} \setminus \{\text{Check}\}, \text{effq: } []\}, \sigma \vdash e \Downarrow_p^\phi s, \pi', \omega \quad \text{Check} \in \pi'.\text{dec} \quad \pi', \sigma \vdash e \Downarrow_p^{\text{Succ}} s', \pi'', \omega' \quad \phi \in \{\text{Init}, \text{Succ}\}}{\pi, \sigma \vdash e \Downarrow_p^\phi s', \pi'', \omega \vdash \omega'}$$

$$\boxed{m, \delta \vdash \text{init}(s) = \langle t, m', \omega \rangle}$$

INITCONST

$$\frac{}{m, \delta \vdash \text{init}(k) = \langle k, m, [] \rangle}$$

INITCLOS

$$\frac{}{m, \delta \vdash \text{init}(cl) = \langle cl, m, [] \rangle}$$

INITARRAY

$$\frac{(m_{i-1}, \delta \vdash \text{init}(s_i) = \langle t_i, m_i, \omega_i \rangle)_{i=1}^n}{m_0, \delta \vdash \text{init}([\bar{s}_i]_{i=1}^n) = \langle [\bar{t}_i]_{i=1}^n, m_n, \vdash_{i=1}^n \omega_i \rangle}$$

INITCOM

$$\frac{\begin{array}{c} m \vdash p \text{ fresh} \\ \delta[C] = \lambda x. e \quad \{ \text{spec: } \langle C, v \rangle, \text{dec: } \{ \}, \text{sttst: } [], \text{effq: } [], \text{child: } \langle \rangle \}, [x \mapsto v] \vdash e \Downarrow_p^{\text{Init}} s, \pi, \omega \\ m[p \mapsto \pi], \delta \vdash \text{init}(s) = \langle t, m', \omega' \rangle \end{array}}{m, \delta \vdash \text{init}(\langle C, v \rangle) = \langle p, m'[p \mapsto \pi\{\text{dec: } \{\text{Effect}\}, \text{child: } t\}], \omega \vdash \omega' \rangle}$$

$$m, \delta \vdash \text{check}(t) = \langle \mu, m', \omega \rangle$$

CHECKCONST

$$\frac{}{m, \delta \vdash \text{check}(k) = \langle \bullet, m, [] \rangle}$$

CHECKCLOS

$$\frac{}{m, \delta \vdash \text{check}(cl) = \langle \bullet, m, [] \rangle}$$

CHECKARRAY

$$\frac{(m_{i-1}, \delta \vdash \text{check}(t_i) = \langle \mu_i, m_i, \omega_i \rangle)_{i=1}^n}{m_0, \delta \vdash \text{check}([\overline{t_i}]_{i=1}^n) = \langle \bigsqcup_{i=1}^n \mu_i, m_n, \uplus_{i=1}^n \omega_i \rangle}$$

CHECKIDLE

$$\frac{m[p] = \pi \quad \text{Check} \notin \pi.\text{dec} \quad m, \delta \vdash \text{check}(\pi.\text{child}) = \langle \mu, m', \omega \rangle}{m, \delta \vdash \text{check}(p) = \langle \mu, m', \omega \rangle}$$

CHECKNOEFFECT

$$\frac{\begin{array}{l} m[p] = \pi \quad \text{Check} \in \pi.\text{dec} \quad \pi.\text{spec} = \langle C, v \rangle \quad \delta[C] = \lambda x. e \\ \pi, [x \mapsto v] \vdash e \Downarrow_p^{\text{Succ}} s, \pi', \omega \quad \text{Effect} \notin \pi'.\text{dec} \quad m, \delta \vdash \text{check}(\pi.\text{child}) = \langle \mu, m', \omega' \rangle \end{array}}{m, \delta \vdash \text{check}(p) = \langle \mu, m' [p \mapsto \pi'], \omega \uplus \omega' \rangle}$$

CHECKEFFECT

$$\frac{\begin{array}{l} m[p] = \pi \quad \text{Check} \in \pi.\text{dec} \quad \pi.\text{spec} = \langle C, v \rangle \quad \delta[C] = \lambda x. e \\ \pi, [x \mapsto v] \vdash e \Downarrow_p^{\text{Succ}} s, \pi', \omega \quad \text{Effect} \in \pi'.\text{dec} \quad m, \delta \vdash \text{reconcile}(\pi.\text{child}, s) = \langle t', m', \omega' \rangle \end{array}}{m, \delta \vdash \text{check}(p) = \langle \otimes, m' [p \mapsto \pi' \{\text{child}: t'\}], \omega \uplus \omega' \rangle}$$

$$m, \delta \vdash \text{reconcile}(t, s) = \langle t', m', \omega \rangle$$

RECONCILEARRAY

$$\frac{(m_{i-1}, \delta \vdash \text{reconcile}(t_i, s_i) = \langle t'_i, m_i, \omega_i \rangle)_{i=1}^n}{m_0, \delta \vdash \text{reconcile}([\overline{t_i}]_{i=1}^n, [\overline{s_i}]_{i=1}^n) = \langle [\overline{t'_i}]_{i=1}^n, m_n, \uplus_{i=1}^n \omega_i \rangle}$$

RECONCILECOMEFFECT

$$\frac{\begin{array}{l} m[p] = \pi \quad \pi.\text{spec} = \langle C, _ \rangle \quad \delta[C] = \lambda x. e \\ \pi \{\text{spec}: \langle C, v \rangle\}, [x \mapsto v] \vdash e \Downarrow_p^{\text{Succ}} s, \pi', \omega \quad m, \delta \vdash \text{reconcile}(\pi.\text{child}, s) = \langle t', m', \omega' \rangle \end{array}}{m, \delta \vdash \text{reconcile}(p, \langle C, v \rangle) = \langle p, m' [p \mapsto \pi' \{\text{dec}: \{\text{Effect}\}, \text{child}: t'\}], \omega \uplus \omega' \rangle}$$

RECONCILECOMNEW

$$\frac{m[p].\text{spec} = \langle C', _ \rangle \quad C \neq C' \quad m, \delta \vdash \text{init}(\langle C, v \rangle) = \langle t', m', \omega \rangle}{m, \delta \vdash \text{reconcile}(p, \langle C, v \rangle) = \langle t', m', \omega \rangle}$$

RECONCILEOTHER

$$\frac{\langle t, s \rangle \neq \langle [\overline{t}], [\overline{s}] \rangle \quad \langle t, s \rangle \neq \langle p, \langle C, v \rangle \rangle \quad m, \delta \vdash \text{init}(s) = \langle t', m', \omega \rangle}{m, \delta \vdash \text{reconcile}(t, s) = \langle t', m', \omega \rangle}$$

$$\begin{array}{c}
 \boxed{m \vdash \text{commitEffs}(t) = \langle m', \omega \rangle} \\
 \\
 \text{COMMITEFFSCONST} \quad \frac{}{m \vdash \text{commitEffs}(k) = \langle m, [] \rangle} \qquad \text{COMMITEFFSCLOS} \quad \frac{}{m \vdash \text{commitEffs}(cl) = \langle m, [] \rangle} \\
 \\
 \text{COMMITEFFSARRAY} \quad \frac{(m_{i-1} \vdash \text{commitEffs}(t_i) = m_i, \omega_i)_{i=1}^n}{m_0 \vdash \text{commitEffs}([\overline{t_i}]_{i=1}^n) = \langle m_n, \text{++}_{i=1}^n \omega_i \rangle} \\
 \\
 \text{COMMITEFFSPATHIDLE} \quad \frac{\text{Effect} \notin m[p].\text{dec} \quad m \vdash \text{commitEffs}(m[p].\text{child}) = \langle m', \omega \rangle}{m \vdash \text{commitEffs}(p) = \langle m', \omega \rangle} \\
 \\
 \text{COMMITEFFSPATH} \quad \frac{\text{Effect} \in m[p].\text{dec} \quad m[p].\text{effq} = [\langle \lambda_{-}.e_i, \sigma_i \rangle]_{i=1}^n \quad m \vdash \text{commitEffs}(m[p].\text{child}) = \langle m_0, \omega_0 \rangle \quad (m_{i-1}, \sigma_i \vdash e_i \Downarrow_{-}^{\text{Normal}} v_i, m_i, \omega_i)_{i=1}^n}{m \vdash \text{commitEffs}(p) = \langle m_n[p \mapsto m_n[p]\{\text{dec}: m_n[p].\text{dec} \setminus \{\text{Effect}\}\}], \text{++}_{i=0}^n \omega_i \rangle}
 \end{array}$$

B Proofs of Theorems

Definition 9 (*e*-Equivalent Views). Views π and π' are *e*-equivalent when the state entries whose labels appear in e are the same. That is,

$$\pi \equiv_e \pi' \iff \forall \ell \in \text{labels}(e), \pi.\text{stst}[\ell] = \pi'.\text{stst}[\ell]. \quad \square$$

Definition 10 (*t*-Equivalent Memories). Tree memories m and m' are *t*-equivalent iff the descendant views of t in m and m' are the same. That is,

$$m \equiv_t m' \iff \forall p \in \text{reachable}(m, t), m[p] = m'[p] \quad \square$$

Lemma 11 (Similar Evaluations). *Evaluations of e in Succ phase with similar views as a context produce identical values and output buffers, along with e-equivalent views. That is, if $\pi, \sigma \vdash e \Downarrow_p^{\text{Succ}} v, \pi', \omega$, then for any $\hat{\pi} \approx \pi$, we have $\hat{\pi}, \sigma \vdash e \Downarrow_p^{\text{Succ}} v, \hat{\pi}', \omega$ where $\hat{\pi}' \equiv_e \pi'$.*

PROOF. Suppose we have $\pi, \sigma \vdash e \Downarrow_p^{\text{Succ}} v, \pi', \omega$ and choose some $\hat{\pi} \approx \pi$. We need to show that $\hat{v} = v$, $\hat{\pi}' \equiv_e \pi'$, and $\hat{\omega} = \omega$, where $\hat{\pi}, \sigma \vdash e \Downarrow_p^{\text{Succ}} \hat{v}, \hat{\pi}', \hat{\omega}$.

We prove that $\hat{v} = v$, $\hat{\pi}' \approx \pi'$ (before showing $\hat{\pi}' \equiv_e \pi'$), and $\hat{\omega} = \omega$ by rule induction. We put a $\hat{\cdot}$ on the intermediate variables in the derivation of $\hat{\pi}, \sigma \vdash e \Downarrow_p^{\text{Succ}} \hat{v}, \hat{\pi}', \hat{\omega}$.

PRINT Evaluations of e produce same values $\hat{v} = v$ and output buffers $\hat{\omega} = \omega$ with similar views $\hat{\pi}' \approx \pi'$ by the IH. Appending the same values to the same output buffers results in the same buffers $\hat{\omega} \mathbin{++} [\hat{v}] = \omega \mathbin{++} [v]$. Resulting values are both $\langle \rangle$. Contexts—views in this case—are unmodified.

APPSETCOMP Evaluations of e_1 and e_2 produce same values $\langle \hat{\ell}, \hat{p} \rangle = \langle \ell, p \rangle$ and $\hat{cl} = cl$, and output buffers $\hat{\omega}_1 = \omega_1$ and $\hat{\omega}_2 = \omega_2$, and similar views $\hat{\pi}_1 \approx \pi_1$ and $\hat{\pi}_2 \approx \pi_2$ by the IH. Concatenations of the same output buffers result in the same buffers $\hat{\omega}_1 \mathbin{++} \hat{\omega}_2 = \omega_1 \mathbin{++} \omega_2$. Appending the same closures $\hat{cl} = cl$ to the state update queues of similar views $\hat{\pi}_2 \approx \pi_2$ produces similar views. (The added **Checks** are discarded anyway during normalization.) Resulting values are both $\langle \rangle$.

STTREBIND Evaluations of the queued e'_i s all produce the intermediate values $\hat{v}_i = v_i$ and output buffers $\hat{\omega}_i = \omega_i$ that are equivalent and views $\hat{\pi}_i \approx \pi_i$ that are similar by the IH. Then the modified $\hat{\pi}_n$ and π_n each modified with the added **Effect** and the same state $\hat{v}_n = v_n$ are still similar. This suffices to apply the IH to the evaluation of e_2 , resulting in the same values $\hat{v} = v$ and output buffers $\hat{\omega}' = \omega'$, and similar views $\hat{\pi}' \approx \pi'$. Thus the resulting values $\hat{v} = v$ and the concatenation of the buffers $(\mathbin{++}_{i=1}^n \hat{\omega}_i) \mathbin{++} \hat{\omega}' = (\mathbin{++}_{i=1}^n \omega_i) \mathbin{++} \omega'$ are equivalent, and the resulting views $\hat{\pi}' \approx \pi'$ are similar.

The remaining cases are trivial.

It is easy to show that if π and $\hat{\pi}$ are similar and *e*-equivalent for some e , then the results are also *e*-equivalent using rule induction.

Now we can prove that $\hat{\pi}' \equiv_e \pi'$, again using rule induction. In each case, evaluation of each sub-expression e' of e produces e' -equivalent views by the IH, and the equivalence is preserved in following evaluations of other sub-expressions. Therefore, the resulting views from π and $\hat{\pi}$ are e' -equivalent for every sub-expression e' , which makes the resulting π' and $\hat{\pi}'$ *e*-equivalent. Note that nested or dead sub-expressions of e do not contain state labels ℓ , due to the syntactic restriction that Hooks can only appear at the top level of a component body.

The only noteworthy case is **STTREBIND** where a state label exists. In this case, the state store entry for ℓ is updated directly to the same value v_n (as just shown above) in both derivations from π and $\hat{\pi}$. Therefore, the resulting π' and $\hat{\pi}'$ have the same state store entry for ℓ . ■

Lemma 7 (Similar Evaluations of Component Body). *Evaluating the component body of similar views produces the same value, view, and output buffer. That is, for π valid under δ where $\pi.\text{spec} = \langle C, v \rangle$*

and $\delta[C] = \langle \lambda x.e, \sigma \rangle$, if $\pi, \sigma[x \mapsto v] \vdash e \Downarrow_p^{\text{Succ}} v', \pi', \omega$, we have $\hat{\pi}, \sigma[x \mapsto v] \vdash e \Downarrow_p^{\text{Succ}} v', \pi', \omega$ for all $\hat{\pi} \approx \pi$.

PROOF. Suppose $\pi, \sigma[x \mapsto v] \vdash e \Downarrow_p^{\text{Succ}} v, \pi', \omega$ and $\hat{\pi}, \sigma[x \mapsto v] \vdash e \Downarrow_p^{\text{Succ}} \hat{v}', \hat{\pi}', \hat{\omega}$, for some π such that $\pi.\text{spec} = \langle C, v \rangle$ where $\delta[C] = \langle \lambda x.e, \sigma \rangle$ and $\hat{\pi} \approx \pi$. By Lemma 11, evaluations of e produce e -equivalent views, that is, the states whose labels appear in e are the same in π' and $\hat{\pi}'$. Since π' and $\hat{\pi}'$ are valid under δ , the only labels that π' and $\hat{\pi}'$ have are those appearing in e . Therefore, $\pi' = \hat{\pi}'$. Similarly, $v' = \hat{v}'$ and $\omega = \hat{\omega}$. Any remaining loops, if present, are evaluated under identical views, trivially producing the same results. ■

Definition 12 (Stability and Semi-Stability). A view π is *stable* iff the re-evaluation of π 's body (as in EVALONCE and EVALMULT) produces the same view as π :

$$\pi\{\text{dec}: \pi.\text{dec} \setminus \{\text{Check}\}, \text{effq}: []\}, \sigma[x \mapsto v] \vdash e \Downarrow_p^\phi _, \pi, \omega$$

where $\pi.\text{spec} = \langle C, v \rangle$ and $\delta[C] = \langle \lambda x.e, \sigma \rangle$.

View π is *semi-stable* iff π with empty state update queues is stable.

We extend this notion to tree memory: m is *t-stable* iff the descendant views of t in m are all stable, and *t-semi-stable* iff the descendant views of t in m are all semi-stable.

Since component body of a stable view can be re-evaluated in an idempotent way, evaluation of the body can be thought of as “reading” the specification of the view. □

Definition 13 (Coherence). A view is *coherent* if its decision is coherent with the status of the state update queue. That is, a coherent view π satisfies the predicate

$$\text{Check} \in \pi.\text{dec} \iff \exists \ell \in \text{dom } \rho, \rho[\ell].\text{sttq} \neq [] \quad \text{where } \rho = \pi.\text{sttst}.$$

We extend this notion to tree memory, i.e., m is *t-coherent* iff the descendants of t in m are coherent. □

Lemma 14 (Preservation of Validity). If $\langle e, \delta \rangle \hookrightarrow^* \langle t, m, \omega, \delta, \mu \rangle$, then m is valid under δ .

PROOF. The only rule that adds states is STTBIND, which appears only in the derivation of *init* where the context being a fresh view. Therefore, it is sufficient to check that the evaluation of the component body in *init* produces a valid view. Due to the syntactic restriction on the usage of Hooks, every **useState** Hook is executed during the evaluation, adding its state labeled ℓ to the view. This process adds every label in the expression to the view, producing a valid view. ■

Lemma 15 (Semi-Stability and Normalized Form). Let π be a semi-stable view valid under δ , and let $\hat{\pi} = \text{normalize}(\pi)$. If $\text{Check} \notin \hat{\pi}.\text{dec}$, then the evaluation of the body produces a view exactly the same as the normalized form. That is, $\pi\{\text{dec}: \pi.\text{dec} \setminus \{\text{Check}\}, \text{effq}: []\}, \sigma[x \mapsto v] \vdash e \Downarrow_p^{\text{Succ}} _, \hat{\pi}, \omega$ where $\pi.\text{spec} = \langle C, v \rangle$ and $\delta[C] = \langle \lambda x.e, \sigma \rangle$.

PROOF. The normalized view $\hat{\pi}$ is the same as π except that the decision and state update queues are empty— $\text{Check} \notin \hat{\pi}.\text{dec}$ implies also $\text{Effect} \notin \hat{\pi}.\text{dec}$ from Definition 4. Thus $\hat{\pi}$ is stable as π is semi-stable. By Definition 12, evaluation with $\hat{\pi}$ produces $\hat{\pi}$ again, which also holds for retrying evaluation. Since $\pi \approx \hat{\pi}$, by Lemma 7, $\pi' = \hat{\pi}$. ■

Lemma 16 (Stability of Retrying Evaluation). Let $\pi.\text{spec} = \langle C, v \rangle$ and $\delta[C] = \langle \lambda x.e, \sigma \rangle$. If π is semi-stable and $\pi, \sigma[x \mapsto v] \vdash e \Downarrow_p^\phi v', \pi', \omega$, then π' is stable.

PROOF. It is sufficient to show that the last evaluation of the component body produces a stable view. Since $\text{Check} \notin \pi'.\text{dec}$, APPSETCOMP is not included in the derivation and all state updates in π are pure. This means $\text{normalize}(\pi)$ has empty queues. Since $\pi \approx \text{normalize}(\pi)$, by the Lemma 15, $\pi' = \text{normalize}(\pi)$, which is stable. ■

Lemma 17 (Preservation of Semi-Stability). *If $\langle e, \delta \rangle \hookrightarrow^* \langle t, m, \omega, \delta, \mu \rangle$, then m is t -semi-stable.*

PROOF. It is sufficient to show that the initial step generates a semi-stable tree memory, and derivation of each step preserves the semi-stability.

STEPINIT, STEPCheck By Lemma 16, evaluation of each component body produces stable view. Updating views' children or adding decisions does not break the stability, so views modified by *init* and *visit* are always stable.

STPEFFECT, STEPEvent The only rule modifying the view during normal evaluation is APPSET-NORMAL. It only adds state updates or modify the decision of a view, which preserves semi-stability of the view. ■

Remark. While the semi-stability of memory is always preserved, the stability is often violated. When setter closures are applied during the execution of Effects or event handlers, the update functions are queued, which are flushed and applied during the next check. Note that the memory is indeed always stable after the check.

Lemma 18 (Preservation of Coherence). *If $\langle e, \delta \rangle \hookrightarrow^* \langle t, m, \omega, \delta, \mu \rangle$, then m is t -coherent. That is, *init* and *visit* always produce t -coherent tree memory.*

PROOF. It is sufficient to show that the initial step generates a coherent tree memory, and the derivation of each step preserves the coherence.

STEPINIT A retrying evaluation of the component body produces a view whose decision does not include **Check** and state queues are empty, thus coherent. The result of *init* only contains new views. Therefore, the resulting memory m is t -coherent.

STEPCheck As stated above, the views whose bodies are evaluated are coherent. Views that are not evaluated are left as is in m' , thus preserving coherence. Therefore, each coherence of the views in the resulting memory m' is preserved.

STPEFFECT, STEPEvent The only rule modifying the view during normal evaluation is APPSET-NORMAL. It adds a state update and adds **Check** decision to the view, which preserves the coherence of the view. ■

Lemma 19 (Similar Reconciliations). *Reconciling with similar memories produces the equivalent results. That is, if $m \vdash \text{reconcile}(t, s) = \langle t', m', \omega \rangle$, then for $\hat{m}$ such that $m \approx_t \hat{m}$, $\hat{m} \vdash \text{reconcile}(t, s) = \langle t', \hat{m}', \omega \rangle$ and $m' \equiv_{t'} \hat{m}'$.*

PROOF. We proceed with induction on the derivation of $m \vdash \text{reconcile}(t, s) = \langle t', m', \omega \rangle$.

RECONCILECOMNEW, RECONCILEOTHER *init*(s) does not read or modify existing views in m ; it only adds new views to m . All new views are descendants of t' , and are the same in m' and $\hat{m}'$. Since the evaluation contexts are identical, the output buffers are also the same.

RECONCILELIST Each derivation of $\text{reconcile}(t_i, s_i)$ produces the same tree, the same output buffer, and t'_i -equivalent memories. Therefore, the final trees and the output buffers are the same and the final memories are $[\bar{t}'_i]_{i=1}^n$ -equivalent.

RECONCILECOMEffect The derivation of $\mathbb{I}_p^\phi$ produces the same v , π' , and ω by Lemma 7 and $\text{reconcile}(t, s)$ produces the same tree and output buffer with t' -equivalent memories by IH. Therefore, the final memories are also t' -equivalent. ■

Lemma 20 (Similar Checks). *Checking with similar memories produces the equivalent results. That is, for similar memories m and $\hat{m}$ that are t -coherent and t -semi-stable and have no view with **Effect** decision set, if $m, \delta \vdash \text{check}(t) = \langle \mu, m', \omega \rangle$, then $\hat{m}, \delta \vdash \text{check}(t) = \langle \mu, \hat{m}', \omega' \rangle$ and $m' \equiv_{t'} \hat{m}'$. We also have $\omega = \omega'$ if component bodies do not print.*

PROOF. We proceed with induction on the derivation of $m, \delta \vdash \text{check}(t) = \langle \mu, m', \omega \rangle$.

CHECKCONST, CHECKCLOS These cases are trivial.

CHECKLIST ($t = [\bar{t}_i]_{i=1}^n$). Each derivation of $\text{check}(\hat{t}_i)$ produces memory, t'_k -equivalent to m_i for $1 \leq k \leq i$ and t'_k -similar to m_i for $i < k \leq n$ by the IHs. As a result, m_n and $\hat{m}_n$ are t'_k -equivalent for $1 \leq k \leq n$, therefore $[\bar{t}'_i]_{i=1}^n$ -equivalent.

CHECKIDLE ($t = p, m[p] = \pi$, and $\hat{m}[p] = \hat{\pi}$). Note that π is coherent and **Check** $\notin \pi.\text{dec}$, implying that the state update queues are empty. Hence, $\text{normalize}(\pi) = \pi$.

- **Check** $\notin \hat{\pi}.\text{dec}$: By the IH, $\text{check}(t)$ produces t -equivalent results.
- **Check** $\in \hat{\pi}.\text{dec}$: By Lemma 15, $\hat{\pi}' = \text{normalize}(\hat{\pi})$. Since $\pi \approx \hat{\pi}$, $\text{normalize}(\hat{\pi}) = \text{normalize}(\pi)$, we have $\hat{\pi}' = \pi$. Since no view—including π —has **Effect** in m , **Effect** $\notin \hat{\pi}'.\text{dec}$ as well. Therefore, $\hat{m}, \delta \vdash \text{check}(p) = _$ is derived from CHECKNOEFFECT, and thus $\hat{m}, \delta \vdash \text{check}(t) = \langle b, \hat{m}', \omega \rangle$. By the IH, $m' \equiv_t \hat{m}'$. As a result, the results are p -equivalent. Note that we have $\omega = \omega'$ if the component prints nothing.

CHECKNOEFFECT ($t = p, m[p] = \pi$, and $\hat{m}[p] = \hat{\pi}$).

- **Check** $\notin \hat{\pi}.\text{dec}$: The proof is similar to the case of CHECKIDLE where **Check** $\in \hat{m}[p].\text{dec}$, except that m and $\hat{m}$ are swapped.
- **Check** $\in \hat{\pi}.\text{dec}$: By Lemma 7, $\hat{\pi}' = \pi'$ (and $\omega = \omega'$ if componets print nothing), and by the IH, $m' \equiv_p \hat{m}'$. Therefore, the results are p -equivalent.

CHECKEFFECT ($t = p$). Since $\text{normalize}(\hat{\pi}) = \text{normalize}(\pi)$ and **Check** $\in \text{normalize}(\pi).\text{dec}$, **Check** $\in \hat{\pi}.\text{dec}$. By Lemma 7, $\pi' = \hat{\pi}'$, and by Lemma 19, $m' \equiv_p \hat{m}'$. Therefore, the results are p -equivalent. ■

Theorem 8 (Similar Transitions). *If a program transitions to a state in check mode $\cup$ during execution, replacing it with a similar state results in the same final state as the original transition. That is, if $\langle e, \delta \rangle \hookrightarrow^* \langle t, m, \omega, \delta, \cup \rangle \hookrightarrow \langle t, m', \omega', \delta, \mu \rangle$, then for any $\hat{m}$ such that $m \approx_t \hat{m}$, we have $\langle t, \hat{m}, \omega, \delta, \cup \rangle \hookrightarrow \langle t, m', \omega'', \delta, \mu \rangle$. We also have $\omega' = \omega''$ when the component bodies do not print.*

PROOF. Since the views not reachable from t are the same in m and $\hat{m}$ before the transition, and these views remain unchanged, it suffices to show that the memories m' and $\hat{m}'$ resulting from the transition from m and $\hat{m}$, respectively, are t -equivalent. The transition from check mode $\cup$ is derived by the *check* rules, so we can apply Lemma 20 if we can show that the views in $\cup$ have no **Effect** decision. Note that m 's t -semi-stability and t -coherence have already been shown in Lemmas 17 and 18.

It also follows from Lemma 20 that we have $\omega' = \omega''$ if component bodies do not print.

We show that only views reachable from the root can contain an **Effect** decision, and such views never contain an **Effect** in check mode $\cup$ or event loop mode $\bullet$.

STEPINIT Every view added by *init* is reachable from the root.

STPEFFECT The *commitEfts* rules remove all **Effect** decisions from reachable views.

STEPCHECK Only paths that are passed as arguments to *check* or returned by *reconcile* may have their views contain an **Effect** decision, and those views are all reachable after *check* or *reconcile*. If any view receives an **Effect** decision, the *check* rule produces the rendered mode $\otimes$, to which the state transitions.

STPEVENT The children and decision fields are not modified during the **Normal** phase. ■

C Comparison of Reactive UI Frameworks

The categorization of reactive GUI web frameworks based on three properties—(a) whether they re-read component specifications for re-rendering, (b) how state updates are processed (queued or immediate), and (c) which reactivity primitives they employ—are given in Table 2.

Table 2. Comparison of reactive UI frameworks.

Framework	Re-read Spec.	State Updates	Reactivity Primitives
React	Yes	Queued	Hooks
Preact	Yes	Queued	Hooks
Dioxus	Yes	Immediate	Signals
Solid	No	Immediate	Signals
Leptos	No	Immediate	Signals
Angular	No	Immediate	Signals
Vue	No	Immediate	Signals
Svelte w/ runes	No	Immediate	Signals
Svelte w/o runes	No	Immediate	Compiler-assisted
SwiftUI	Yes	Immediate	Compiler-assisted

We compare various reactive UI frameworks to

- (1) check if state updates are queued or immediate, and
- (2) check if component logic gets re-evaluated every render.

C.1 React

counter gets printed every render. count updates are queued.

```

1 import { useState, useEffect } from 'react';
2 function Counter() {
3   console.log('counter');
4   const [count, setCount] = useState(0);
5   useEffect(() => {
6     if (count >= 3) {
7       setCount(0);
8     }
9   }, [count]);
10  function h() {
11    console.log(count);
12    setCount(count + 1);
13    console.log(count);
14  }
15  return (
16    <button onClick={h}>
17      {count}
18    </button>
19  );
20 }
```

C.2 Preact

counter gets printed every render. count updates are queued.

```

1 import { useState, useEffect } from 'preact/hooks';
2 function Counter() {
3   console.log('counter');
4   const [count, setCount] = useState(0);
5   useEffect(() => {
```

```

6   if (count >= 3) {
7     setCount(0);
8   }
9 }, [count]);
10 function h() {
11   console.log(count);
12   setCount(count + 1);
13   console.log(count);
14 }
15 return (
16   <button onClick={h}>
17     {count}
18   </button>
19 );
20 }

```

C.3 Dioxus

counter gets printed every render. count updates are immediate.

```

1 use dioxus::prelude::*;
2 fn log(msg: &str) { ... }
3 #[component]
4 fn Counter() -> Element {
5   log("Counter");
6   let mut count = use_signal(|| 0);
7   use_effect(move || {
8     if count() >= 3 {
9       count.set(0);
10    }
11  });
12  rsx! {
13    button {
14      onclick: move |_| {
15        log(count.to_string().as_str());
16        count += 1;
17        log(count.to_string().as_str());
18      },
19      "{count}"
20    }
21  }
22 }

```

C.4 Solid

counter gets printed only once. count updates are immediate.

```

1 import { createSignal, createEffect } from 'solid-js';
2 function Counter() {
3   console.log('counter');
4   const [count, setCount] = createSignal(0);
5   createEffect(() => {
6     if (count() >= 3) {
7       setCount(0);
8     }
9   });
10  function h() {
11    console.log(count());
12    setCount(count() + 1);
13    console.log(count());

```



```

14 }
15 return (
16   <button onClick={h}>
17     {count()}
18   </button>
19 );
20 }

```

C.5 Leptos

counter gets printed only once. count updates are immediate.

```

1 use leptos::leptos_dom::logging::*;
2 use leptos::prelude::*;
3 #[component]
4 pub fn Counter() -> impl IntoView {
5   console_log("Counter");
6   let (value, set_value) = signal(0);
7   Effect::new(move |_| {
8     if value.get() >= 3 {
9       set_value.set(0);
10    }
11  });
12   view! {
13     <button on:click=move |_| {
14       console_log(&value.get().to_string());
15       set_value.update(|value| *value += 1);
16       console_log(&value.get().to_string());
17     }>{value}</button>
18   }
19 }

```

C.6 Angular

counter gets printed only once. count updates are immediate.

```

1 import {Component, signal, effect} from '@angular/core';
2 @Component({
3   selector: 'app-root',
4   template: `
5     <button (click)="h()">
6       {{ count() }}
7     </button>
8   `,
9 })
10 export class CounterComponent {
11   count = signal(0);
12   constructor() {
13     console.log('counter');
14     effect(() => {
15       if (this.count() >= 3) {
16         this.count.set(0);
17       }
18     });
19   }
20   h() {
21     console.log(this.count());
22     this.count.update(val => val + 1);
23     console.log(this.count());
24   }
25 }

```

25 }

C.7 Vue

counter gets printed only once. count updates are immediate.

```

1 <script setup>
2 import { ref, watch } from 'vue';
3 console.log('counter');
4 const count = ref(0);
5 watch(count, (newValue) => {
6   if (newValue >= 3) {
7     count.value = 0;
8   }
9 });
10 function h() {
11   console.log(count.value);
12   count.value += 1;
13   console.log(count.value);
14 }
15 </script>
16 <template>
17   <button @click="h">
18     {{ count }}
19   </button>
20 </template>

```

C.8 Svelte

C.8.1 Svelte with Runes. counter gets printed only once. count updates are immediate.

```

1 <script>
2   console.log('counter');
3   let count = $state(0);
4   $effect(() => {
5     if (count >= 3) {
6       count = 0;
7     }
8   });
9   function h() {
10    console.log(count);
11    count += 1;
12    console.log(count);
13  }
14 </script>
15 <button onclick={h}>
16   {count}
17 </button>

```

C.8.2 Svelte without Runes. counter gets printed only once. count updates are immediate.

```

1 <svelte:options runes={false} />
2 <script>
3   console.log('counter');
4   let count = 0;
5   $: if (count >= 3) {
6     count = 0;
7   }
8   function h() {
9     console.log(count);
10    count += 1;

```

```
11     console.log(count);
12 }
13 </script>
14 <button on:click={h}>
15   {count}
16 </button>
```

C.9 SwiftUI

counter gets printed every render. count updates are immediate.

```
1 struct Counter: View {
2   @State private var count = 0
3   var body: some View {
4     print("counter")
5     return Button("\(count)") {
6       print(count)
7       count += 1
8       print(count)
9     }.onChange(of: count) { oldValue, newValue in
10       if newValue >= 3 {
11         count = 0
12       }
13     }
14   }
15 }
```

Received 2025-03-25; accepted 2025-08-12